

Guia 1

Quick Finnance

Documentação do Sistema - v 1.0

Aluna: Wanessa Luna e Diego Barbosa

Escopo do Projeto: Quick Finnance (QFin)

1. Introdução

Este documento apresenta os requisitos iniciais para a concepção de um Sistema de Gestão Financeira Pessoal na forma de um website, denominado Quick Finnance (QFin). O objetivo geral é fornecer aos usuários uma plataforma intuitiva para gerenciar suas finanças pessoais, incluindo o registro de receitas, despesas, e informações sobre financiamentos.

1.1 Objetivo Geral do Documento

Este documento tem informações dos requisitos iniciais para concepção de um Sistema de Controle de Finanças Pessoais.

1.2 Propósito do Sistema (Escopo)

- a) Identificar o sistema por um nome e sigla: QFin – Quick Finnance.
- b) O Sistema QFin visa gerenciar e controlar as finanças pessoais, permitindo que os usuários insiram suas despesas, ganhos, e informações sobre financiamentos, proporcionando uma visão clara de sua situação financeira.

1.3 Visão Geral

Este documento está disposto numa forma linear e as seções do mesmo são:

Escopo do Projeto: Quick Finnance (QFin)	1
1. Introdução	1
1.1 Objetivo Geral do Documento	1
1.2 Propósito do Sistema (Escopo)	1
1.3 Visão Geral	1
1.4. Definições, Siglas e Abreviações	3
2. Informações do Cliente	3
2.1. Patrocinadores do Produto	3
2.2. Usuários do Sistema	3
2.3. Interessados no Sucesso do Sistema	3
3. Características do Sistema	5
3.1. Requisitos Funcionais	5
3.2. Requisitos Não Funcionais	9
Requisitos de Performance (RPER)	9
Requisitos de Usabilidade (RUSA)	9
Requisitos de Plataforma de Hardware (RPHW)	9
Requisitos de Plataforma de Software (RPSW)	10
Requisitos de Portabilidade (RPOR)	11
Requisitos de Disponibilidade (RDISP)	11
Requisitos de Segurança (RSEG)	12
Requisitos de Manutenibilidade (RMAN)	12

Outros Requisitos Não-Funcionais (RNFO).....	13
4. Glossário.....	13
5. Casos de Uso.....	14
5.1 Visão Geral.....	14
5.1.1 Módulos Identificados.....	14
5.2 Boas Práticas Aplicadas.....	14
5.2.1 Organização Modular.....	14
5.2.2 Notação UML Padrão.....	15
5.2.3 Detalhamento Adequado.....	15
5.2.4 Legibilidade.....	15
5.3.1 Objetivo.....	16
5.3.2 Atores.....	16
5.3.3 Casos de Uso.....	16
5.3.4 Regras de Negócio.....	18
5.4 Módulo de Transações.....	19
5.4.1 Objetivo.....	19
5.4.2 Atores.....	19
5.4.3 Casos de Uso.....	20
5.4.4 Regras de Negócio.....	21
5.5 Módulo de Financiamentos.....	22
5.5.1 Objetivo.....	22
5.5.2 Atores.....	22
5.5.3 Casos de Uso.....	22
5.5.4 Regras de Negócio.....	24
5.6 Módulo de Metas Financeiras.....	25
5.6.1 Objetivo.....	25
5.6.2 Atores.....	25
5.6.3 Casos de Uso.....	25
5.6.4 Tipos de Meta.....	26
5.6.5 Regras de Negócio.....	26
5.7 Módulo de Relatórios.....	27
5.7.1 Objetivo.....	27
5.7.2 Atores.....	27
5.7.3 Casos de Uso.....	28
5.7.4 Regras de Negócio.....	29
5.8 Módulo Dashboard.....	30
5.8.1 Objetivo.....	30
5.8.2 Atores.....	30
5.8.3 Casos de Uso.....	30
5.8.4 Regras de Negócio.....	31
5.9 Relacionamentos e Dependências.....	31
5.9.1 Relacionamentos Include (<<include>>).....	31
5.9.2 Relacionamentos Extend (<<extend>>).....	32
5.10 Considerações de Implementação.....	33

5.10.1	Priorização de Desenvolvimento.....	33
5.10.2	Requisitos Não Funcionais.....	34
5.10.3	Integração entre Módulos.....	35
5.10.4	Testes de Aceitação.....	35
6.	Protótipos.....	36
7.	Modelo de Arquitetura do Sistema.....	39
7.1	Visão Geral da Arquitetura.....	39
7.2	Principais Componentes.....	39
7.3	Arquitetura em Camadas (Layered Architecture).....	40
7.4	Detalhamento dos Módulos.....	41
7.4.1.	Core Service (Spring Boot).....	41
7.4.2.	Auxiliary Services (Node.js).....	42
7.5	Camada de Dados.....	43
7.5.1.	PostgreSQL e Hibernate PostgreSQL:.....	43
7.5.2.	Redis.....	43
7.6	Padrões de Comunicação.....	44
7.6.1.	Comunicação Síncrona:.....	44
7.6.2.	Comunicação Assíncrona:.....	44
8.	Diagramas de Classes Detalhados.....	44
8.1	Diagrama de Entidades JPA.....	44
8.2	Diagrama de Serviços e Repositórios.....	46
8.3	Padrões de Design.....	46
8.4	Diagrama de Controladores REST.....	47
8.4.1.	Estrutura e Anotações.....	47
8.5.	Diagrama de DTOs (Data Transfer Objects).....	47
8.5.1.	Tipos de DTOs.....	48
9.	Diagramas de Sequência / Colaboração.....	49
9.1	Diagrama de Sequência - Login do Usuário.....	49
9.1.1.	Descrição do Fluxo.....	49
9.1.2.	Pontos Chave Segurança:.....	50
9.2	Diagrama de Sequência - Registro de Usuário.....	50
9.2.1.	Descrição do Fluxo.....	51
9.3	Diagrama de Sequência - Criar Transação.....	51
9.3.1.	Descrição do Fluxo.....	52
9.4	Diagrama de Sequência - Listar Transações.....	52
9.4.1.	Descrição do Fluxo.....	52
9.5	Diagrama de Sequência - Criar Financiamento.....	53
9.5.1	Descrição do Fluxo.....	53
9.7	Diagrama de Sequência - Carregar Dashboard.....	53
9.7.1	Descrição do Fluxo.....	54
10.	Modelo de Dados (DER – Diagrama Entidade-Relacionamento).....	54
10.1	Visão Geral do Modelo de Dados.....	55
10.2	Modelo de Dados Detalhado com Constraints e Índices.....	55
10.3	Dicionário de Dados.....	55

10.3.1 Tabela users.....	56
10.3.2 Tabela categories.....	56
10.3.3 Tabela transactions.....	56
10.3.4 Tabela financings.....	57

1.4. Definições, Siglas e Abreviações

- **QFin:** Quick Finnance
- **RF:** Requisito Funcional
- **RPER:** Requisitos de Performance
- **RUSA:** Requisitos de Usabilidade
- **RPHW:** Requisitos de Plataforma de Hardware
- **RPSW:** Requisitos de Plataforma de Software
- **RPOR:** Requisitos de Portabilidade
- **RDISP:** Requisitos de Disponibilidade
- **RSEG:** Requisitos de Segurança
- **RMAN:** Requisitos de Manutenibilidade
- **RNFO:** Outros Requisitos Não-Funcionais

2. Informações do Cliente

2.1. Patrocinadores do Produto

- Wanessa Luna - Aluna
- Diego Barbosa - Aluno
- José Ricardo - Professor
- Universidade Estadual de Goiás - Instituição

2.2. Usuários do Sistema

- Usuário: Pessoa Física
- Função: Usuário Final
- Alocação na Organização: Indivíduo

2.3. Interessados no Sucesso do Sistema

- Interessados: Patrocinadores
- Motivação: Aprendizado.
- Interessados: Usuário Final
- Motivação: Ter controle financeiro, organizar despesas e receitas, planejar o futuro financeiro.

3. Características do Sistema

3.1. Requisitos Funcionais

Código Requisito: RF01

- **Prioridade:** Alta
- **Descrição:** O usuário deve poder fazer lançamentos no software tanto de suas receitas quanto das despesas.
- **Justificativa:** Permitir ao usuário ter controle do quanto ele gasta (despesa) e do quanto ele ganha (receita).
- **Origem:** Cliente (Pessoa Física)
- **Critério de Verificação:** Lista de Lançamentos na tela ou impresso. Formulário de cadastro funcionando.
- **Satisfação do cliente:** Alta
- **Grau de Estabilidade:** Alta
- **Requisitos Necessários:** Nenhum
- **Requisitos Dependentes:** RF02, RF03
- **Conflitos:** Nenhum
- **Materiais de Suporte (Anexos):** Entrevista com o cliente.
- **Histórico:** Criado em 27 de agosto de 2025

Código Requisito: RF02

- **Prioridade:** Alta
- **Descrição:** O usuário deve poder inserir informações detalhadas sobre financiamentos (valor, parcelas, juros, data de vencimento).
- **Justificativa:** Permitir ao usuário acompanhar seus compromissos financeiros de longo prazo.
- **Origem:** Cliente (Pessoa Física)
- **Critério de Verificação:** Lista de financiamentos na tela ou impresso. Formulário de cadastro de financiamento funcionando.
- **Satisfação do cliente:** Alta
- **Grau de Estabilidade:** Alta
- **Requisitos Necessários:** Nenhum
- **Requisitos Dependentes:** Nenhum
- **Conflitos:** Nenhum
- **Materiais de Suporte (Anexos):** Entrevista com o cliente.
- **Histórico:** Criado em 27 de agosto de 2025

Código Requisito: RF03

- **Prioridade:** Média

- **Descrição:** Tanto as receitas quanto as despesas devem ser classificadas em uma categoria e uma subcategoria.
 - **Justificativa:** O usuário deve ter uma forma de organizar seus lançamentos.
 - **Origem:** Gerente da Software House
 - **Critério de Verificação:** Lista de categorias e subcategorias na tela ou impresso.
 - **Satisfação do cliente:** Média
 - **Grau de Estabilidade:** Média
 - **Requisitos Necessários:** RF01
 - **Requisitos Dependentes:** RF04, RF05
 - **Conflitos:** Nenhum
 - **Materiais de Suporte (Anexos):** Anexo - Entrevista.
 - **Histórico:** Criado em 27 de agosto de 2025
-

Código Requisito: RF04

- **Prioridade:** Média
 - **Descrição:** O usuário deve ter a opção de criar suas próprias categorias e subcategorias de receitas e despesas.
 - **Justificativa:** O usuário deve ter uma forma de organizar seus lançamentos de forma que possa ter um maior controle de seus gastos e ganhos.
 - **Origem:** Gerente da Software House
 - **Critério de Verificação:** Lista de categorias e subcategorias na tela ou impresso.
 - **Satisfação do cliente:** Média
 - **Grau de Estabilidade:** Média
 - **Requisitos Necessários:** RF03
 - **Requisitos Dependentes:** Nenhum
 - **Conflitos:** Nenhum
 - **Materiais de Suporte (Anexos):** Anexo - Entrevista.
 - **Histórico:** Criado em 27 de agosto de 2025
-

Código Requisito: RF05

- **Prioridade:** Média
- **Descrição:** O software já deve ter pré-definido as categorias e subcategorias mais comuns, de receitas e despesa.
- **Justificativa:** Agilizar o cadastro de lançamentos quando o usuário não deseja cadastrar nenhuma categoria e subcategoria.
- **Origem:** Gerente da Software House
- **Critério de Verificação:** Lista de categorias e subcategorias na tela ou impresso.
- **Satisfação do cliente:** Média
- **Grau de Estabilidade:** Média
- **Requisitos Necessários:** RF03
- **Requisitos Dependentes:** Nenhum
- **Conflitos:** Nenhum
- **Materiais de Suporte (Anexos):** Anexo - Entrevista.
- **Histórico:** Criado em 27 de agosto de 2025

Código Requisito: RF06

- **Prioridade:** Alta
 - **Descrição:** O sistema deve gerar relatórios e gráficos de receitas e despesas por período e categoria.
 - **Justificativa:** Fornecer ao usuário uma visão clara e visual de sua situação financeira.
 - **Origem:** Cliente (Pessoa Física)
 - **Critério de Verificação:** Visualização de relatórios e gráficos na tela.
 - **Satisfação do cliente:** Alta
 - **Grau de Estabilidade:** Alta
 - **Requisitos Necessários:** RF01, RF03
 - **Requisitos Dependentes:** Nenhum
 - **Conflitos:** Nenhum
 - **Materiais de Suporte (Anexos):** Entrevista com o cliente.
 - **Histórico:** Criado em 27 de agosto de 2025
-

Código Requisito: RF07

- **Prioridade:** Média
 - **Descrição:** O sistema deve permitir a exportação de dados de receitas, despesas e financiamentos para formatos comuns (ex: CSV, PDF).
 - **Justificativa:** Permitir ao usuário utilizar seus dados em outras ferramentas ou para fins de arquivamento.
 - **Origem:** Cliente (Pessoa Física)
 - **Critério de Verificação:** Geração e download de arquivos nos formatos especificados.
 - **Satisfação do cliente:** Média
 - **Grau de Estabilidade:** Média
 - **Requisitos Necessários:** Nenhum
 - **Requisitos Dependentes:** Nenhum
 - **Conflitos:** Nenhum
 - **Materiais de Suporte (Anexos):** Entrevista com o cliente.
 - **Histórico:** Criado em 27 de agosto de 2025
-

Código Requisito: RF08

- **Prioridade:** Média
- **Descrição:** O sistema deve permitir ao usuário definir metas financeiras (ex: economizar X por mês, pagar dívida Y até Z data).
- **Justificativa:** Incentivar o usuário a alcançar seus objetivos financeiros.
- **Origem:** Cliente (Pessoa Física)
- **Critério de Verificação:** Cadastro e acompanhamento de metas na tela.
- **Satisfação do cliente:** Média
- **Grau de Estabilidade:** Média

- **Requisitos Necessários:** Nenhum
 - **Requisitos Dependentes:** Nenhum
 - **Conflitos:** Nenhum
 - **Materiais de Suporte (Anexos):** Entrevista com o cliente.
 - **Histórico:** Criado em 27 de agosto de 2025
-

3.2. Requisitos Não Funcionais

Requisitos de Performance (RPER)

Código Requisito: RPER01

- **Prioridade:** Média
 - **Descrição:** O tempo de carregamento de qualquer página do sistema não deve exceder 3 segundos em uma conexão de internet banda larga padrão.
 - **Justificativa:** Garantir uma experiência de usuário fluida e evitar frustrações.
 - **Origem:** Gerente da Software House
 - **Critério de Verificação:** Testes de performance com ferramentas de monitoramento de tempo de carregamento.
 - **Satisfação do Cliente:** Alta
 - **Grau de Estabilidade:** Alta
 - **Requisitos Necessários:** Nenhum
 - **Requisitos Dependentes:** Nenhum
 - **Conflitos:** Nenhum
 - **Histórico:** Criado em 27 de agosto de 2025
-

Requisitos de Usabilidade (RUSA)

Código Requisito: RUSA01

- **Prioridade:** Alta
 - **Descrição:** A interface do usuário deve ser intuitiva e fácil de usar, mesmo para usuários com pouca experiência em sistemas de gestão financeira.
 - **Justificativa:** Maximizar a adoção e satisfação do usuário.
 - **Origem:** Cliente (Pessoa Física)
 - **Critério de Verificação:** Testes de usabilidade com usuários reais e coleta de feedback.
 - **Satisfação do Cliente:** Alta
 - **Grau de Estabilidade:** Alta
 - **Requisitos Necessários:** Nenhum
 - **Requisitos Dependentes:** RNFO01
 - **Conflitos:** Nenhum
 - **Materiais de Suporte (Anexos):** Anexo A – Entrevista com o patrocinador.
 - **Histórico:** Criado em 27 de agosto de 2025
-

Requisitos de Plataforma de Hardware (RPHW)

Código Requisito: RPHW01

- **Prioridade:** Baixa

- Descrição: O servidor que hospeda o sistema deve ter capacidade para suportar um mínimo de 1000 usuários simultâneos.
 - Justificativa: Garantir a escalabilidade e disponibilidade do sistema para um número crescente de usuários.
 - Origem: Gerente da Software House
 - Critério de Verificação: Testes de carga e monitoramento de recursos do servidor.
 - Satisfação do Cliente: Baixa
 - Grau de Estabilidade: Alta
 - Requisitos Necessários: Nenhum
 - Requisitos Dependentes: Nenhum
 - Conflitos: Nenhum
 - Materiais de Suporte (Anexos): Anexo A – Entrevista com o patrocinador.
 - Histórico: Criado em 20 de maio de 2024
-

Requisitos de Plataforma de Software (RPSW)

Código Requisito: RPSW01

- Prioridade: Baixa
 - Descrição: O software deverá ser desenvolvido utilizando tecnologias web modernas (ex: React, Angular, Vue.js para frontend e Node.js, Python/Django, Ruby on Rails para backend).
 - Justificativa: Adoção de padrões de mercado para garantir manutenibilidade, escalabilidade e acesso a uma vasta comunidade de desenvolvedores.
 - Origem: Gerente da Software House
 - Critério de Verificação: Verificação do código e da arquitetura do sistema.
 - Satisfação do Cliente: Baixa
 - Grau de Estabilidade: Alta
 - Requisitos Necessários: Nenhum
 - Requisitos Dependentes: Nenhum
 - Conflitos: Nenhum
 - Materiais de Suporte (Anexos): Anexo A – Entrevista com o patrocinador.
 - Histórico: Criado em 27 de agosto de 2025
-

Código Requisito: RPSW02

- Prioridade: Baixa
- Descrição: O software deverá ter como SGDB um banco de dados relacional (ex: PostgreSQL, MySQL) ou NoSQL (ex: MongoDB), dependendo da análise de requisitos de escalabilidade e tipo de dados.
- Justificativa: A justificativa para esta restrição é que esta é plataforma-padrão adotada pela empresa ou a mais adequada para o escopo do projeto.
- Origem: Gerente da Software House
- Critério de Verificação: Verificação da configuração do banco de dados.
- Satisfação do Cliente: Baixa
- Grau de Estabilidade: Alta

- Requisitos Dependentes: Nenhum
 - Conflitos: Nenhum
 - Materiais de Suporte (Anexos): Anexo A – Entrevista com o patrocinador.
 - Histórico: Criado em 27 de agosto de 2025
-

Requisitos de Portabilidade (RPOR)

Código Requisito: RPOR01

- Prioridade: Baixa
 - Descrição: Do ponto de vista do cliente, o site deve rodar em qualquer browser web moderno (Chrome, Firefox, Safari, Edge).
 - Justificativa: Facilitar o uso de qualquer usuário em qualquer browser.
 - Origem: Gerente da Software House
 - Critério de Verificação: Testes de compatibilidade em diferentes navegadores.
 - Satisfação do Cliente: Média
 - Grau de Estabilidade: Alta
 - Requisitos Dependentes: Nenhum
 - Conflitos: Nenhum
 - Materiais de Suporte (Anexos): Anexo A – Entrevista com o patrocinador.
 - Histórico: Criado em 27 de agosto de 2025
-

Requisitos de Disponibilidade (RDISP)

Código Requisito: RDISP01

- Prioridade: Média
 - Descrição: Todas as operações não poderão ficar indisponíveis por mais de vinte e quatro horas.
 - Justificativa: A indisponibilidade pode acarretar descrédito do sistema ao usuário.
 - Origem: Gerente da Software House
 - Critério de Verificação: Monitoramento contínuo da disponibilidade do sistema.
 - Satisfação do Cliente: Alta
 - Grau de Estabilidade: Alta
 - Requisitos Dependentes: Nenhum
 - Conflitos: Nenhum
 - Materiais de Suporte (Anexos): Anexo A – Entrevista com o patrocinador.
 - Histórico: Criado em 27 de agosto de 2025
-

Requisitos de Segurança (RSEG)

Código Requisito: RSEG01

- Prioridade: Alta
 - Descrição: O software não deve permitir o acesso indevido às informações dos usuários. As mesmas deverão ser acessadas por login e senha.
 - Justificativa: Permitir privacidade e segurança aos usuários do sistema.
 - Origem: Gerente da Software House, Cliente (Pessoa Física)
 - Critério de Verificação: Testes de segurança (penetration testing), auditorias de código.
 - Satisfação do Cliente: Alta
 - Grau de Estabilidade: Alta
 - Requisitos Dependentes: Nenhum
 - Conflitos: Nenhum
 - Materiais de Suporte (Anexos): Anexo A – Entrevista com o patrocinador.
 - Histórico: Criado em 27 de agosto de 2025
-

Código Requisito: RSEG02

- Prioridade: Alta
 - Descrição: Todas as comunicações entre o cliente e o servidor devem ser criptografadas (HTTPS).
 - Justificativa: Proteger a integridade e confidencialidade dos dados do usuário durante a transmissão.
 - Origem: Gerente da Software House
 - Critério de Verificação: Verificação do certificado SSL/TLS e inspeção de tráfego de rede.
 - Satisfação do Cliente: Alta
 - Grau de Estabilidade: Alta
 - Requisitos Dependentes: Nenhum
 - Conflitos: Nenhum
 - Materiais de Suporte (Anexos): Anexo A – Entrevista com o patrocinador.
 - Histórico: Criado em 27 de agosto de 2025
-

Requisitos de Manutenibilidade (RMAN)

Código Requisito: RMAN01

- Prioridade: Média
- Descrição: O código-fonte do sistema deve ser bem documentado e seguir padrões de codificação para facilitar a manutenção e futuras modificações.
- Justificativa: Reduzir o custo e o tempo de manutenção do software.
- Origem: Gerente da Software House
- Critério de Verificação: Revisão de código e auditoria de documentação.

- Satisfação do Cliente: Baixa
 - Grau de Estabilidade: Alta
 - Requisitos Necessários: Nenhum
 - Requisitos Dependentes: Nenhum
 - Conflitos: Nenhum
 - Materiais de Suporte (Anexos): Anexo A – Entrevista com o patrocinador.
 - Histórico: Criado em 27 de agosto de 2025
-

Outros Requisitos Não-Funcionais (RNFO)

Código Requisito: RNFO01

- Prioridade: Média
 - Descrição: O software deve prover ajuda on-line sensível ao contexto em todas as suas telas.
 - Justificativa: Permitir a apreensibilidade do software.
 - Origem: Gerente da Software House, Cliente (Pessoa Física)
 - Critério de Verificação: Testes de usabilidade e verificação da funcionalidade de ajuda em todas as telas.
 - Satisfação do Cliente: Alta
 - Grau de Estabilidade: Alta
 - Requisitos Dependentes: Nenhum
 - Conflitos: Nenhum
 - Materiais de Suporte (Anexos): Anexo A – Entrevista com o patrocinador.
 - Histórico: Criado em 27 de agosto de 2025
-

4. Glossário

- Conta: Conta bancária cadastrada no sistema.
- Lançamento: Débito ou Crédito adicionado em uma conta cadastrada.
- Baixar Lançamento: Alterar a situação do lançamento de a pagar para pago.
- Categoria: Agrupamento de várias subcategorias.
- Subcategoria: Agrupamento de lançamentos.
- Filtro de consulta: Usado para configurar a visualização dos dados seja pelo relatório, seja pelo gráfico.
- Relatório: Estrutura de visualização de dados, organizado por linhas e colunas.
- Internauta: Qualquer usuário de internet.
- E-mail: Correio-Eletrônico, utilizado para receber mensagens eletrônicas via internet.
- Período base: É o período escolhido pelo usuário para visualização das informações do sistema.
- Financiamento: Empréstimo de longo prazo com parcelas e juros.

5. Casos de Uso

5.1 Visão Geral

Os diagramas de casos de uso do sistema QFin foram organizados em 6 módulos distintos, cada um focado em uma área específica de funcionalidade. Esta abordagem modular facilita:

Compreensão: Cada diagrama é focado e não sobrecarregado

Manutenção: Alterações podem ser feitas em módulos específicos

Desenvolvimento: Equipes podem trabalhar em paralelo

Documentação: Facilita a criação de especificações detalhadas

5.1.1 Módulos Identificados

1. **Autenticação:** Login, logout e recuperação de senha
2. **Transações:** Gestão de receitas e despesas
3. **Financiamentos:** Controle de empréstimos e financiamentos
4. **Metas Financeiras:** Definição e acompanhamento de objetivos
5. **Relatórios:** Geração e exportação de análises
6. **Dashboard:** Visão geral e resumos financeiros

5.2 Boas Práticas Aplicadas

5.2.1 Organização Modular

- **Separação por Domínio:** Cada diagrama representa um domínio específico
- **Baixo Acoplamento:** Minimização de dependências entre módulos
- **Alta Coesão:** Casos de uso relacionados agrupados

5.2.2 Notação UML Padrão

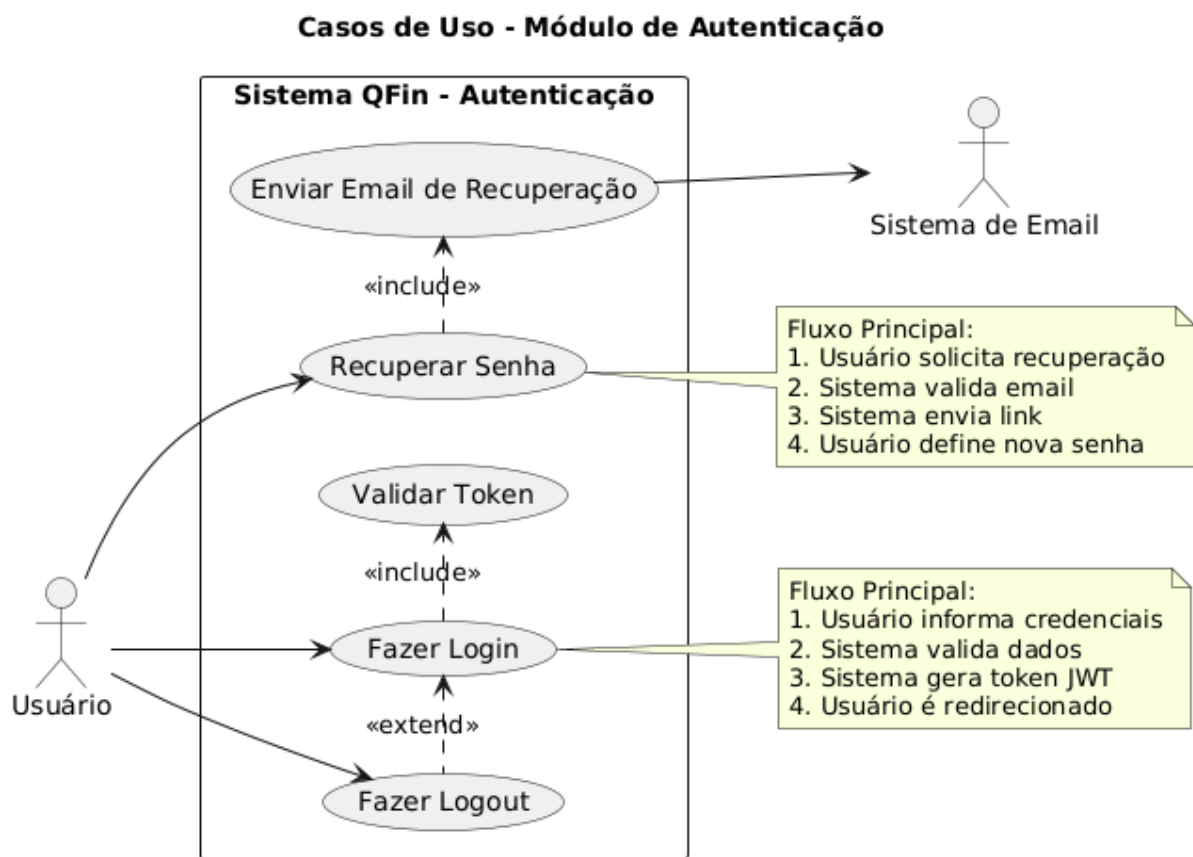
- **Atores Primários:** Usuário principal do sistema
- **Atores Secundários:** Sistemas externos (email, notificações)
- **Relacionamentos:** Include, extend e associações claramente definidos
- **Documentação:** Notas explicativas para casos complexos

5.2.3 Detalhamento Adequado

- **Fluxos Principais:** Descritos em notas quando necessário
- **Pré e Pós-condições:** Especificadas para casos críticos
- **Exceções:** Tratadas através de relacionamentos extend

5.2.4 Legibilidade

- **Layout:** Orientação left-to-right para melhor leitura
- **Nomenclatura:** Nomes claros e descritivos
- **Agrupamento:** Casos relacionados visualmente próximos



5.3 Módulo de Autenticação

5.3.1 Objetivo

Gerenciar o acesso seguro dos usuários ao sistema QFin.

5.3.2 Atores

- Usuário: Ator principal que interage com o sistema
- Sistema de Email: Ator secundário para recuperação de senha

5.3.3 Casos de Uso

UC01 - Fazer Login

- Descrição: Permite ao usuário acessar o sistema com credenciais válidas
- Pré-condição: Usuário deve possuir conta cadastrada
- Fluxo Principal:
 1. Usuário informa email e senha
 2. Sistema valida credenciais
 3. Sistema gera token JWT
 4. Usuário é redirecionado para dashboard
- Pós-condição: Usuário autenticado no sistema
- Inclui: Validar Token

UC02 - Fazer Logout

- Descrição: Permite ao usuário sair do sistema com segurança
- Pré-condição: Usuário deve estar autenticado
- Fluxo Principal:
 5. Usuário solicita logout
 6. Sistema invalida token
 7. Sistema redireciona para tela de login
- Pós-condição: Sessão encerrada
- Estende: Fazer Login (logout automático após inatividade)

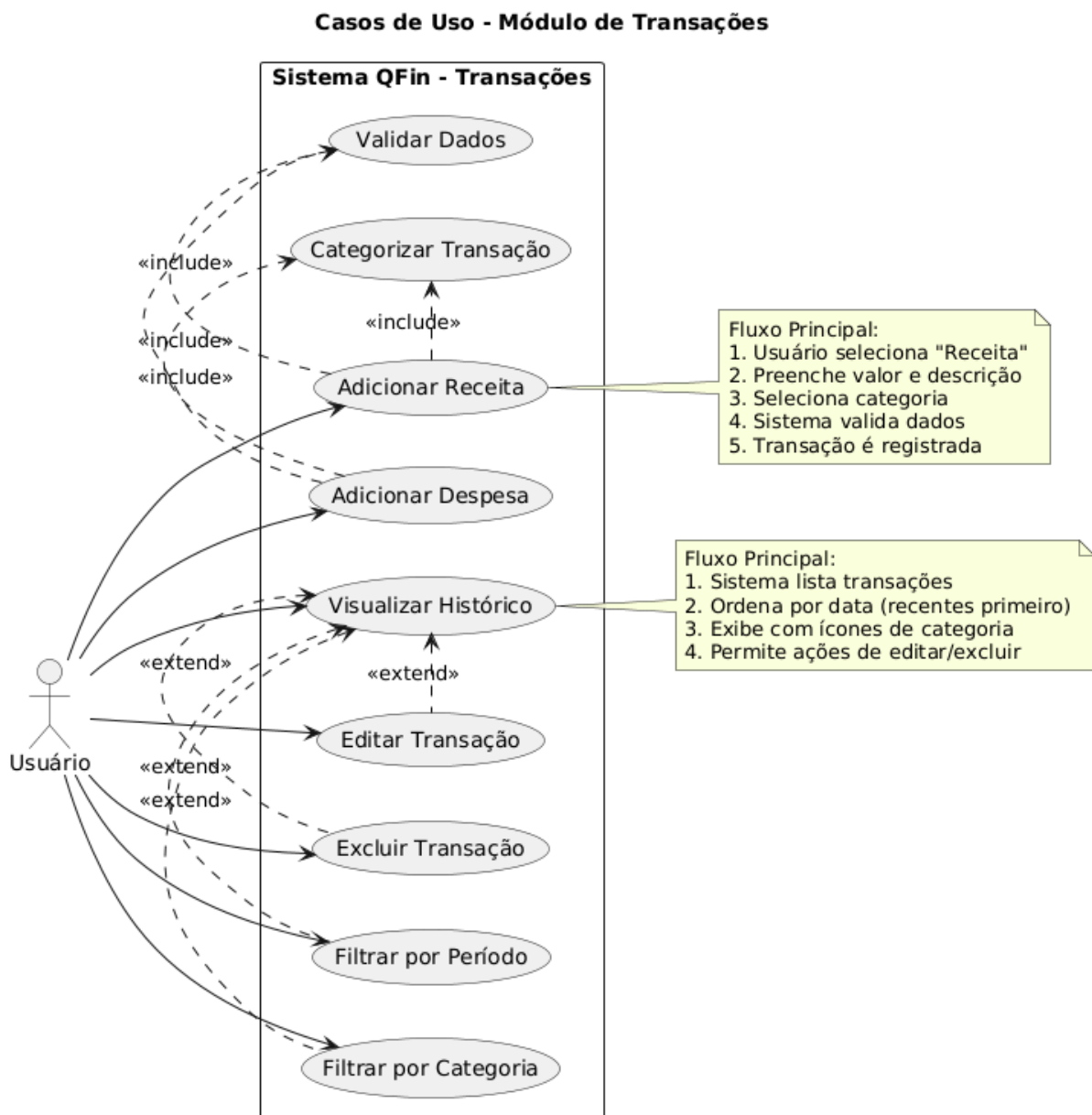
UC03 - Recuperar Senha

- Descrição: Permite recuperar acesso quando senha é esquecida
- Pré-condição: Email deve estar cadastrado no sistema
- Fluxo Principal:
 8. Usuário solicita recuperação de senha
 9. Sistema valida email
 10. Sistema envia link de recuperação
 11. Usuário define nova senha
- Pós-condição: Nova senha definida

- Inclui: Enviar Email de Recuperação

5.3.4 Regras de Negócio

- Tokens JWT expiram em 24 horas
- Máximo 3 tentativas de login incorretas
- Links de recuperação expiram em 1 hora
- Senhas devem ter mínimo 8 caracteres



5.4 Módulo de Transações

5.4.1 Objetivo

Gerenciar todas as transações financeiras (receitas e despesas) do usuário.

5.4.2 Atores

- Usuário: Ator principal que gerencia suas transações

5.4.3 Casos de Uso

UC01 - Adicionar Receita

- Descrição: Permite cadastrar uma nova receita
- Pré-condição: Usuário autenticado
- Fluxo Principal:
 1. Usuário seleciona tipo "Receita"
 2. Preenche valor, descrição e data
 3. Seleciona categoria
 4. Sistema valida dados
 5. Transação é registrada
- Pós-condição: Receita cadastrada e saldo atualizado
- Inclui: Categorizar Transação, Validar Dados

UC02 - Adicionar Despesa

- Descrição: Permite cadastrar uma nova despesa
- Pré-condição: Usuário autenticado
- Fluxo Principal: Similar à receita, mas tipo "Despesa"
- Pós-condição: Despesa cadastrada e saldo atualizado
- Inclui: Categorizar Transação, Validar Dados

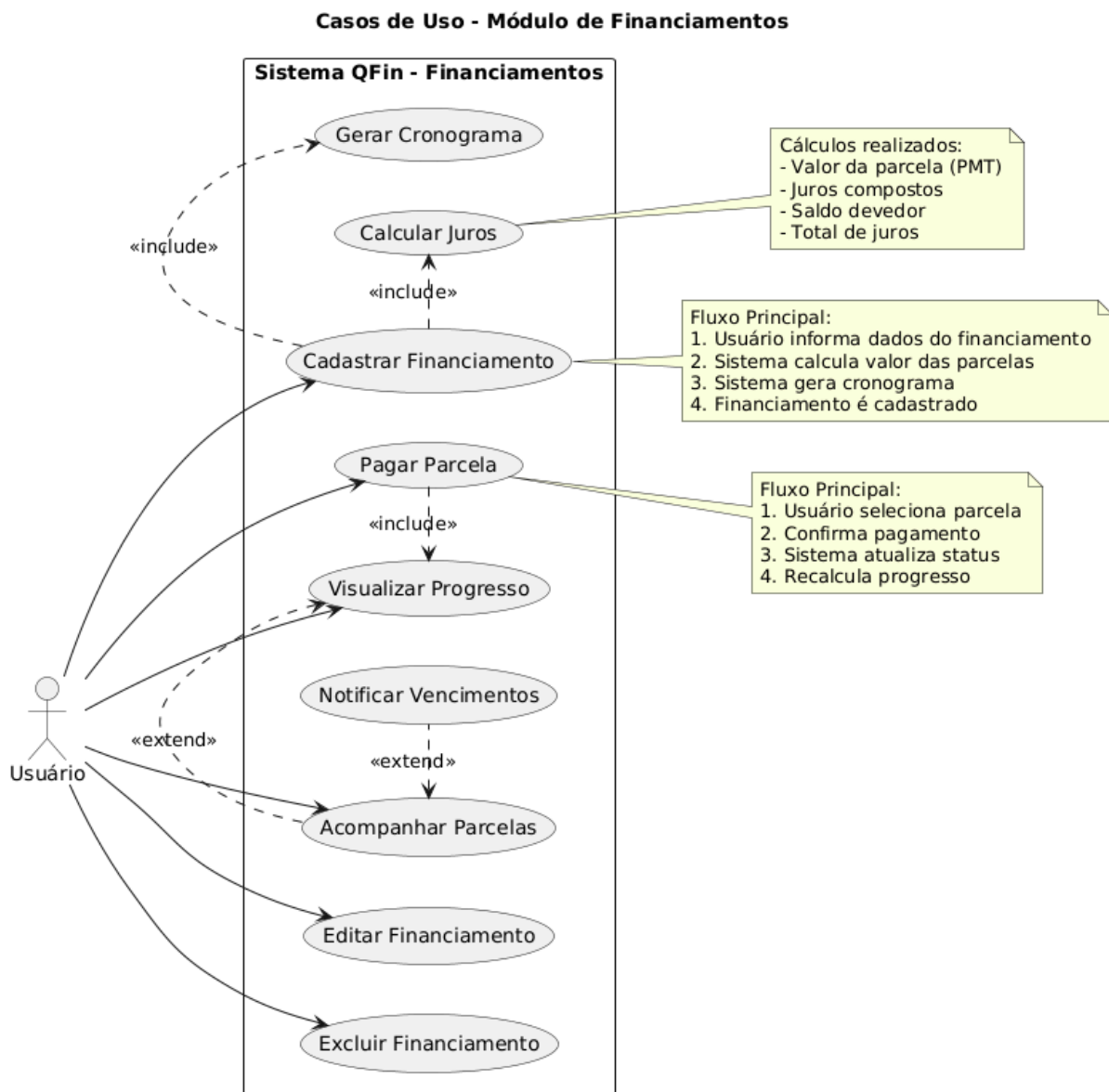
UC05 - Visualizar Histórico

- Descrição: Exibe lista de todas as transações do usuário
- Pré-condição: Usuário autenticado
- Fluxo Principal:
 1. Sistema recupera transações do usuário
 2. Ordena por data (mais recentes primeiro)
 3. Exibe lista com ícones de categoria
 4. Permite ações de editar/excluir

- Pós-condição: Histórico exibido
- Estendido por: Editar Transação, Excluir Transação, Filtrar por Período

5.4.4 Regras de Negócio

- Valores devem ser positivos
- Data não pode ser futura
- Descrição é obrigatória
- Categoria deve ser válida para o tipo



5.5 Módulo de Financiamentos

5.5.1 Objetivo

Controlar financiamentos, empréstimos e acompanhar pagamento de parcelas.

5.5.2 Atores

- Usuário: Ator principal que gerencia financiamentos

5.5.3 Casos de Uso

UC01 - Cadastrar Financiamento

- Descrição: Registra um novo financiamento no sistema
- Pré-condição: Usuário autenticado
- Fluxo Principal:
 1. Usuário informa dados do financiamento
 2. Sistema calcula valor das parcelas
 3. Sistema gera cronograma de pagamentos
 4. Financiamento é cadastrado
- Pós-condição: Financiamento cadastrado com cronograma
- Inclui: Calcular Juros, Gerar Cronograma

UC03 - Pagar Parcela

- Descrição: Registra o pagamento de uma parcela
- Pré-condição: Financiamento cadastrado com parcelas pendentes
- Fluxo Principal:
 1. Usuário seleciona parcela a pagar
 2. Confirma pagamento
 3. Sistema atualiza status da parcela
 4. Sistema recalcula progresso
- Pós-condição: Parcela marcada como paga
- Inclui: Visualizar Progresso

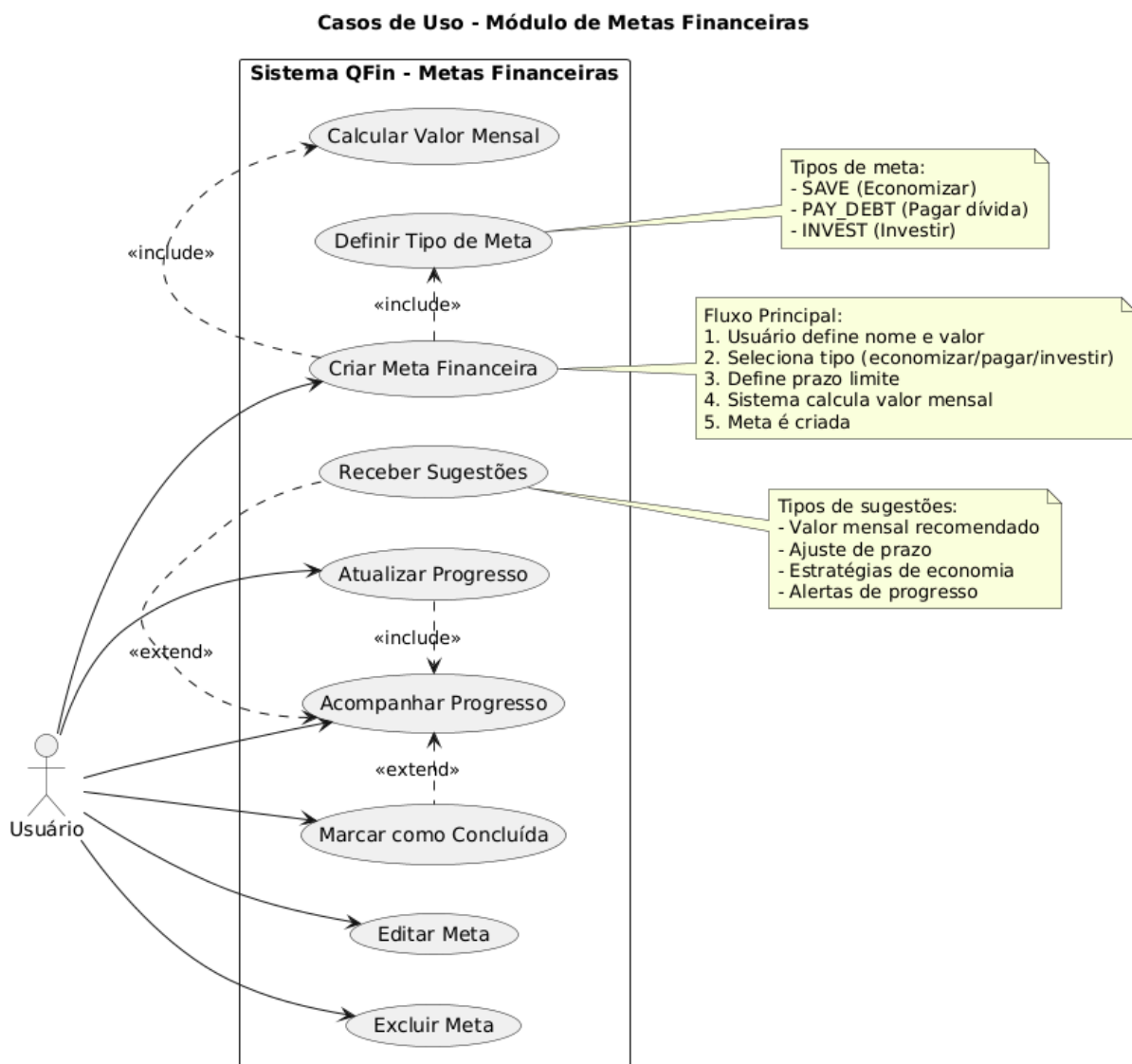
UC05 - Calcular Juros

- Descrição: Realiza cálculos financeiros do financiamento
- Cálculos Realizados:
 - Valor da parcela (PMT)
 - Juros compostos

- Saldo devedor
- Total de juros a pagar

5.5.4 Regras de Negócio

- Taxa de juros deve ser positiva
- Número de parcelas deve ser maior que zero
- Primeira parcela não pode ser no passado
- Valor total deve ser positivo



5.6 Módulo de Metas Financeiras

5.6.1 Objetivo

Permitir que usuários definam e acompanhem objetivos financeiros.

5.6.2 Atores

- Usuário: Ator principal que define e acompanha metas

5.6.3 Casos de Uso

UC01 - Criar Meta Financeira

- Descrição: Cadastra uma nova meta financeira
- Pré-condição: Usuário autenticado
- Fluxo Principal:
 1. Usuário define nome e valor da meta
 2. Seleciona tipo (economizar/pagar dívida/investir)
 3. Define prazo limite
 4. Sistema calcula valor mensal necessário
 5. Meta é criada
- Pós-condição: Meta cadastrada com sugestões
- Inclui: Definir Tipo de Meta, Calcular Valor Mensal

UC05 - Receber Sugestões

- Descrição: Fornece recomendações personalizadas
- Tipos de Sugestões:
 - Valor mensal recomendado
 - Ajuste de prazo
 - Estratégias de economia
 - Alertas de progresso
- Estende: Acompanhar Progresso

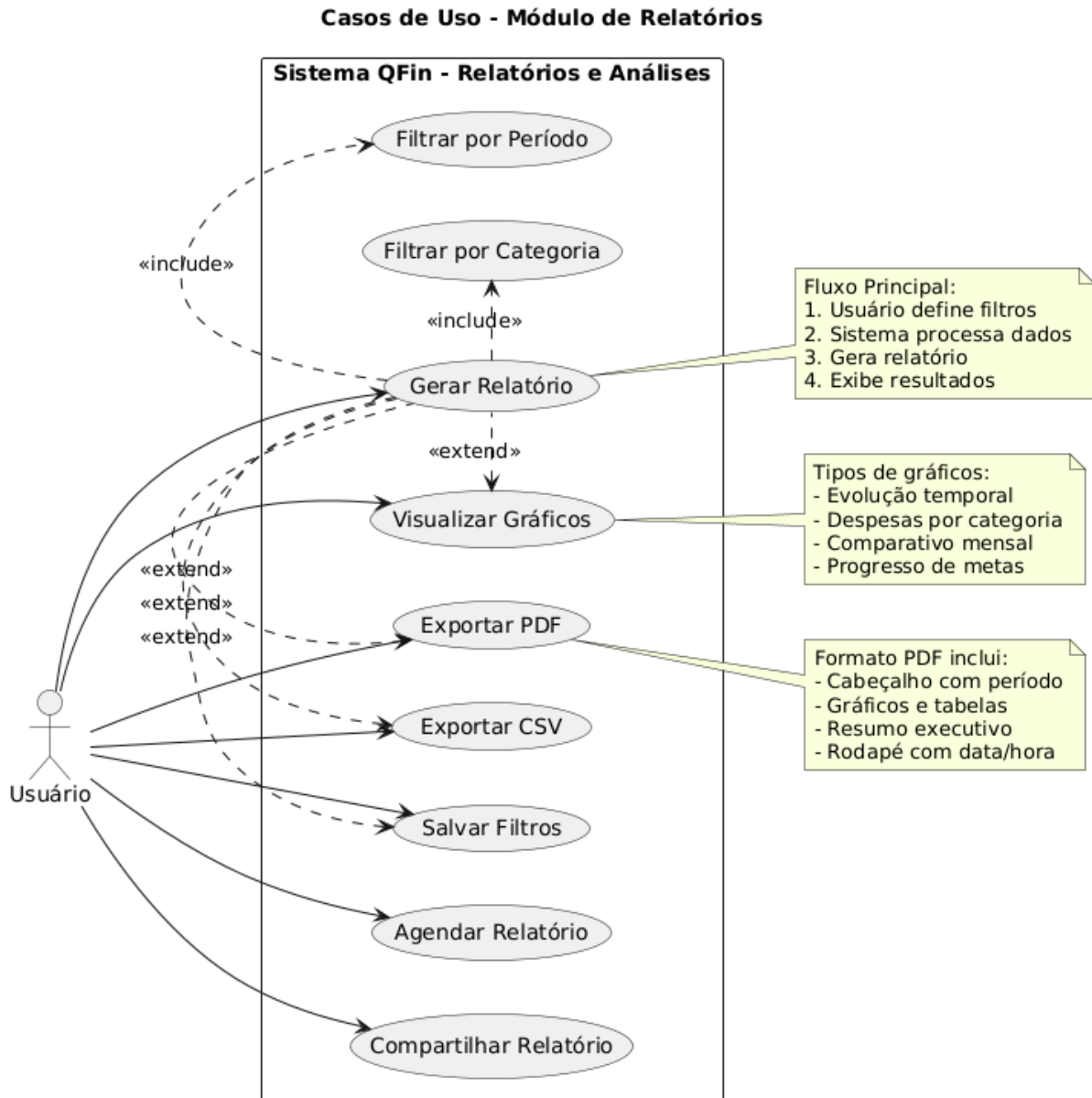
5.6.4 Tipos de Meta

- SAVE: Economizar para um objetivo
- PAY_DEBT: Pagar uma dívida específica
- INVEST: Juntar dinheiro para investimento

5.6.5 Regras de Negócio

- Valor objetivo deve ser positivo
- Data limite deve ser futura
- Valor atual não pode ser negativo

- Nome da meta é obrigatório



5.7 Módulo de Relatórios

5.7.1 Objetivo

Gerar análises e relatórios personalizados dos dados financeiros.

5.7.2 Atores

- Usuário: Ator principal que gera e consome relatórios

5.7.3 Casos de Uso

UC01 - Gerar Relatório

- Descrição: Cria relatórios personalizados
- Pré-condição: Usuário com dados cadastrados
- Fluxo Principal:
 1. Usuário define filtros desejados
 2. Sistema processa dados conforme filtros
 3. Sistema gera relatório
 4. Exibe resultados
- Pós-condição: Relatório gerado
- Inclui: Filtrar por Período, Filtrar por Categoria

UC04 - Exportar PDF

- Descrição: Exporta relatório em formato PDF
- Formato PDF Inclui:
 - Cabeçalho com período selecionado
 - Gráficos e tabelas
 - Resumo executivo
 - Rodapé com data/hora de geração
- Estende: Gerar Relatório

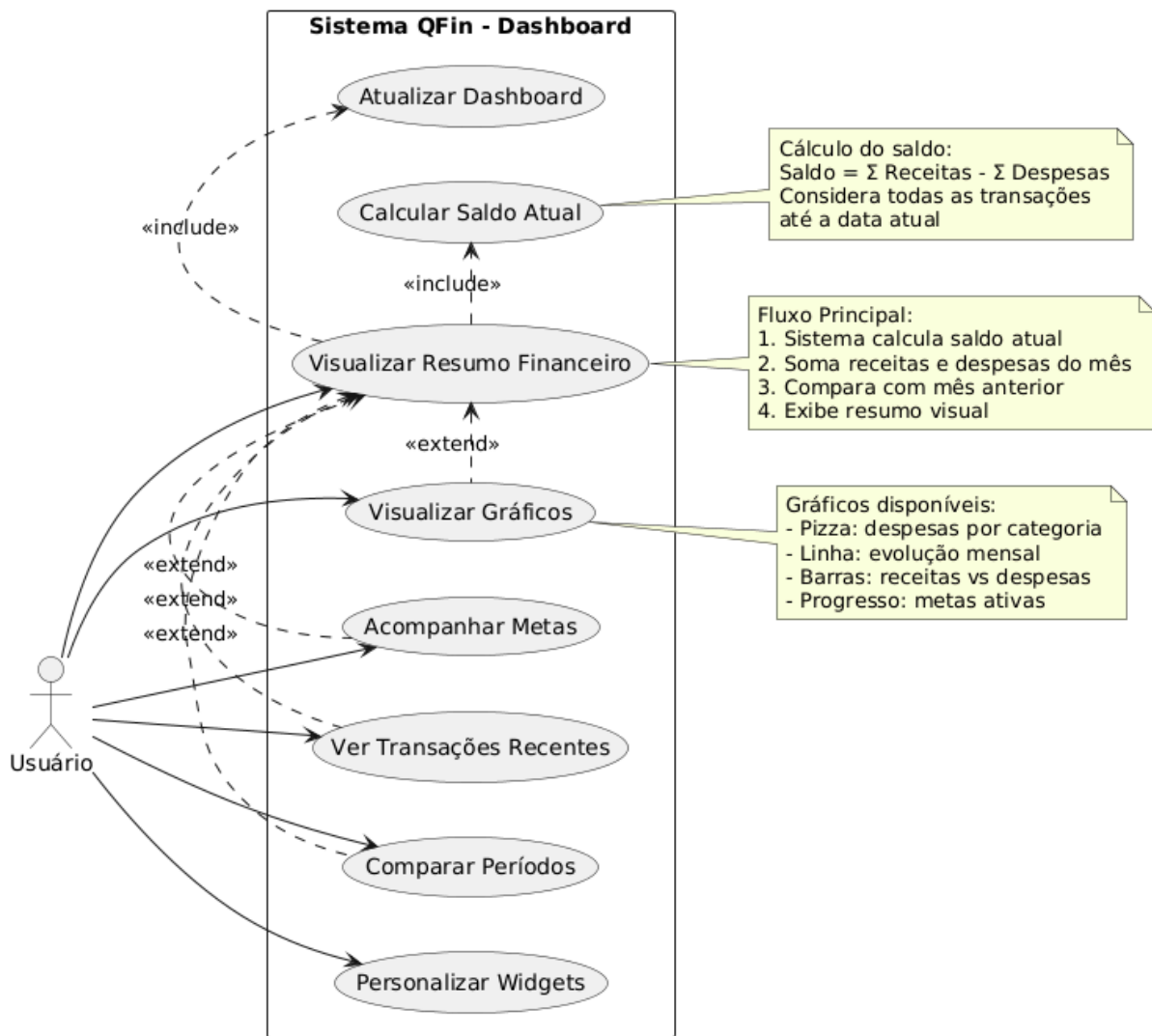
UC06 - Visualizar Gráficos

- Tipos de Gráficos:
 - Evolução temporal das finanças
 - Despesas por categoria
 - Comparativo mensal
 - Progresso de metas
- Estende: Gerar Relatório

5.7.4 Regras de Negócio

- Relatórios só incluem dados do usuário logado
- Período máximo de 2 anos
- Exportação limitada a 10 relatórios por dia
- Arquivos são mantidos por 30 dias

Casos de Uso - Módulo Dashboard



5.8 Módulo Dashboard

5.8.1 Objetivo

Fornecer uma visão geral e consolidada da situação financeira do usuário.

5.8.2 Atores

- Usuário: Ator principal que visualiza o dashboard

5.8.3 Casos de Uso

UC01 - Visualizar Resumo Financeiro

- Descrição: Exibe visão geral da situação financeira
- Pré-condição: Usuário autenticado
- Fluxo Principal:
 1. Sistema calcula saldo atual
 2. Soma receitas e despesas do mês
 3. Compara com mês anterior
 4. Exibe resumo visual
- Pós-condição: Resumo financeiro exibido
- Inclui: Calcular Saldo Atual, Atualizar Dashboard

UC02 - Visualizar Gráficos

- Gráficos Disponíveis:
 - Pizza: despesas por categoria
 - Linha: evolução mensal
 - Barras: receitas vs despesas
 - Progresso: metas ativas
- Estende: Visualizar Resumo Financeiro

UC05 - Calcular Saldo Atual

- Fórmula: $\text{Saldo} = \sum \text{Receitas} - \sum \text{Despesas}$
- Considera: Todas as transações até a data atual
- Atualização: Em tempo real a cada nova transação

5.8.4 Regras de Negócio

- Dashboard atualiza automaticamente
- Dados são do mês atual por padrão
- Gráficos limitados a últimos 12 meses
- Cache de 5 minutos para performance

5.9 Relacionamentos e Dependências

5.9.1 Relacionamentos Include (<<include>>)

Autenticação

- Login → Validar Token
- Recuperar Senha → Enviar Email

Transações

- Adicionar Receita → Categorizar Transação
- Adicionar Despesa → Categorizar Transação
- Adicionar Receita → Validar Dados
- Adicionar Despesa → Validar Dados

Financiamentos

- Cadastrar Financiamento → Calcular Juros
- Cadastrar Financiamento → Gerar Cronograma
- Pagar Parcela → Visualizar Progresso

Metas

- Criar Meta → Definir Tipo de Meta
- Criar Meta → Calcular Valor Mensal
- Atualizar Progresso → Acompanhar Progresso

Relatórios

- Gerar Relatório → Filtrar por Período
- Gerar Relatório → Filtrar por Categoria

Dashboard

- Visualizar Resumo → Calcular Saldo Atual
- Visualizar Resumo → Atualizar Dashboard

5.9.2 Relacionamentos Extend (<<extend>>)

Autenticação

- Fazer Login ← Fazer Logout

Transações

- Visualizar Histórico ← Editar Transação
- Visualizar Histórico ← Excluir Transação

- Visualizar Histórico ← Filtrar por Período
- Visualizar Histórico ← Filtrar por Categoria

Financiamentos

- Acompanhar Parcelas ← Notificar Vencimentos
- Visualizar Progresso ← Acompanhar Parcelas

Metas

- Acompanhar Progresso ← Receber Sugestões
- Acompanhar Progresso ← Marcar como Concluída

Relatórios

- Gerar Relatório ← Visualizar Gráficos
- Gerar Relatório ← Exportar PDF
- Gerar Relatório ← Exportar CSV
- Gerar Relatório ← Salvar Filtros

Dashboard

- Visualizar Resumo ← Visualizar Gráficos
- Visualizar Resumo ← Acompanhar Metas
- Visualizar Resumo ← Ver Transações Recentes
- Visualizar Resumo ← Comparar Períodos

5.10 Considerações de Implementação

5.10.1 Priorização de Desenvolvimento

Fase 1 - MVP (Minimum Viable Product)

1. Autenticação (Login/Logout)
2. Transações básicas (Adicionar/Visualizar)
3. Dashboard simples

Fase 2 - Funcionalidades Intermediárias

1. Categorização avançada
2. Metas financeiras básicas
3. Relatórios simples

Fase 3 - Funcionalidades Avançadas

1. Financiamentos completos
2. Relatórios avançados com gráficos
3. Sugestões inteligentes

5.10.2 Requisitos Não Funcionais

Performance

- Dashboard deve carregar em menos de 2 segundos
- Relatórios devem ser gerados em menos de 10 segundos
- Sistema deve suportar 1000 usuários simultâneos

Segurança

- Autenticação obrigatória para todos os casos de uso
- Dados financeiros criptografados
- Logs de auditoria para operações críticas

Usabilidade

- Interface responsiva para mobile e desktop
- Máximo 3 cliques para qualquer funcionalidade
- Feedback visual para todas as ações

5.10.3 Integração entre Módulos

Dependências de Dados

- Dashboard depende de Transações e Metas
- Relatórios dependem de todos os módulos
- Metas podem ser atualizadas por Transações

Eventos do Sistema

- Nova transação → Atualizar dashboard
- Meta atingida → Notificar usuário
- Parcela vencida → Enviar alerta

5.10.4 Testes de Aceitação

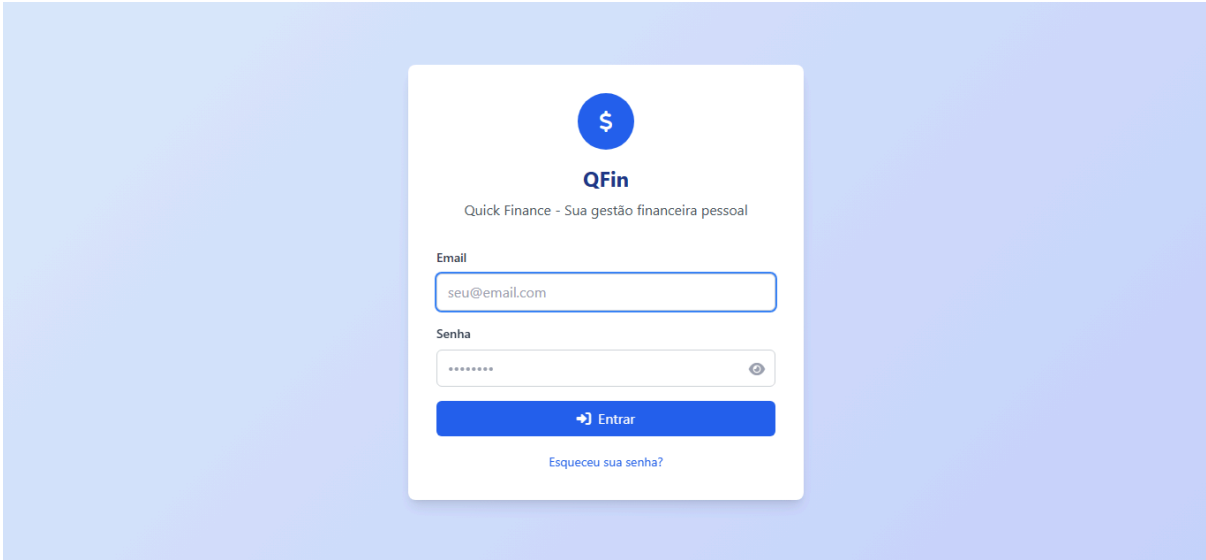
Critérios por Módulo

- Autenticação: Login bem-sucedido em menos de 3 segundos
- Transações: Cadastro de transação em menos de 30 segundos
- Financiamentos: Cálculo correto de parcelas
- Metas: Sugestões precisas baseadas em dados reais
- Relatórios: Exportação sem erros em múltiplos formatos
- Dashboard: Dados sempre atualizados e precisos

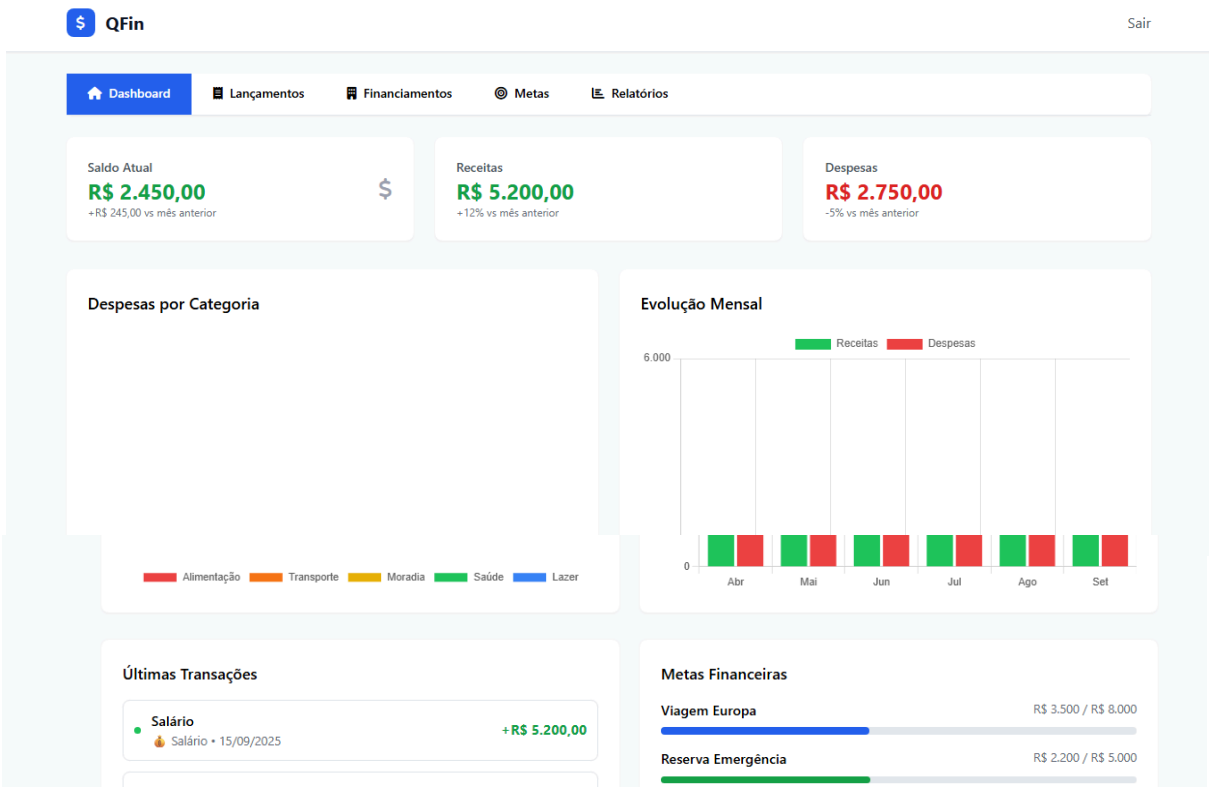
6. Protótipos

Segue a prototipação inicial do sistema:

Tela de Login:



Tela Inicial (Dashboard):



Tela para lançamento de receitas ou despesas:

QFin

Sair

Dashboard

Lançamentos

Financiamentos

Metas

Relatórios

Nova Transação

Tipo

Receita

Valor

0,00

Descrição

Descrição da transação

Categoria

Alimentação

Data

dd/mm/aaaa

+ Adicionar Transação

Histórico de Transações

Salário

Salário • 15/09/2025

+ R\$ 5.200,00

Supermercado

Alimentação • 14/09/2025

- R\$ 320,50

Tela para cadastro e visualização de financiamentos:

QFin

Sair

Dashboard

Lançamentos

Financiamentos

Metas

Relatórios

Novo Financiamento

Nome do Financiamento

Ex: Financiamento do Carro

Valor Total

0,00

Número de Parcelas

12

Taxa de Juros (% a.m.)

1,5

Data da Primeira Parcela

dd/mm/aaaa

+ Adicionar Financiamento

Financiamento do Carro

Progresso

8/48 parcelas

Valor da Parcela

R\$ 520,00

Valor Restante

R\$ 28.800,00

Taxa de Juros

1,2% a.m.

Próximo Vencimento

25/09/2025

Financiamento da Casa

Progresso

120/360 parcelas

Valor da Parcela

R\$ 1.250,00

Valor Restante

R\$ 300.000,00

Taxa de Juros

0,8% a.m.

Próximo Vencimento

10/10/2025

Tela para definição de metas financeiras:

Dashboard

Lançamentos

Financiamentos

Metas

Relatórios

Nova Meta Financeira

Nome da Meta

Ex: Viagem para Europa

Tipo de Meta

Economizar

Valor Alvo

0,00

Valor Atual

0,00

Data Limite

dd/mm/aaaa

+ Adicionar Meta

Viagem Europa

Meta de Economia

Progresso

43,8%

Valor Atual

R\$ 3.500,00

Valor Alvo

R\$ 8.000,00

Restante

R\$ 4.500,00

Prazo

180 dias

Sugestão: Economize R\$ 750,00 por mês para atingir sua meta.

Reserva de Emergência

Meta de Economia

Progresso

44,0%

Valor Atual

R\$ 2.200,00

Valor Alvo

R\$ 5.000,00

Restante

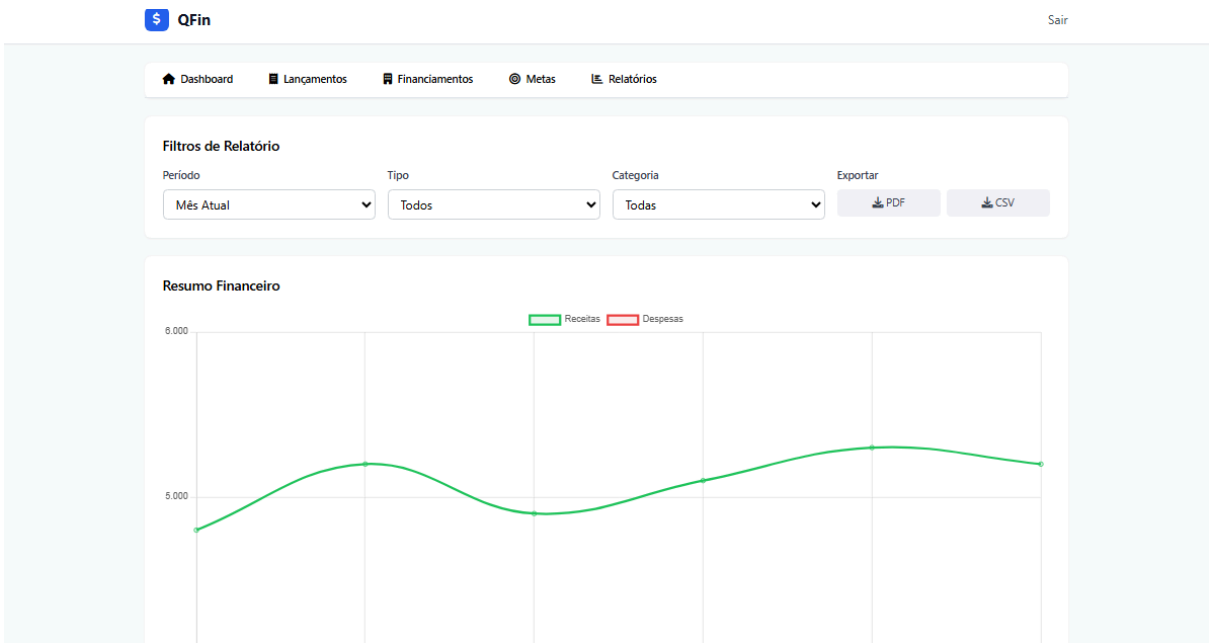
R\$ 2.800,00

Prazo

365 dias

Sugestão: Economize R\$ 230,00 por mês para atingir sua meta.

Tela para visualizar e exportar os relatórios:



7. Modelo de Arquitetura do Sistema

7.1 Visão Geral da Arquitetura

A arquitetura adotada é um modelo híbrido, combinando uma abordagem de monolito modular para o serviço principal com micro serviços para funcionalidades auxiliares e assíncronas. Esta abordagem permite a robustez e a simplicidade de um monolito para as operações centrais, ao mesmo tempo que oferece a flexibilidade e a escalabilidade dos micro serviços para tarefas específicas como notificações e geração de relatórios. O diagrama abaixo ilustra a visão geral da arquitetura, mostrando a interação entre os principais componentes do sistema.

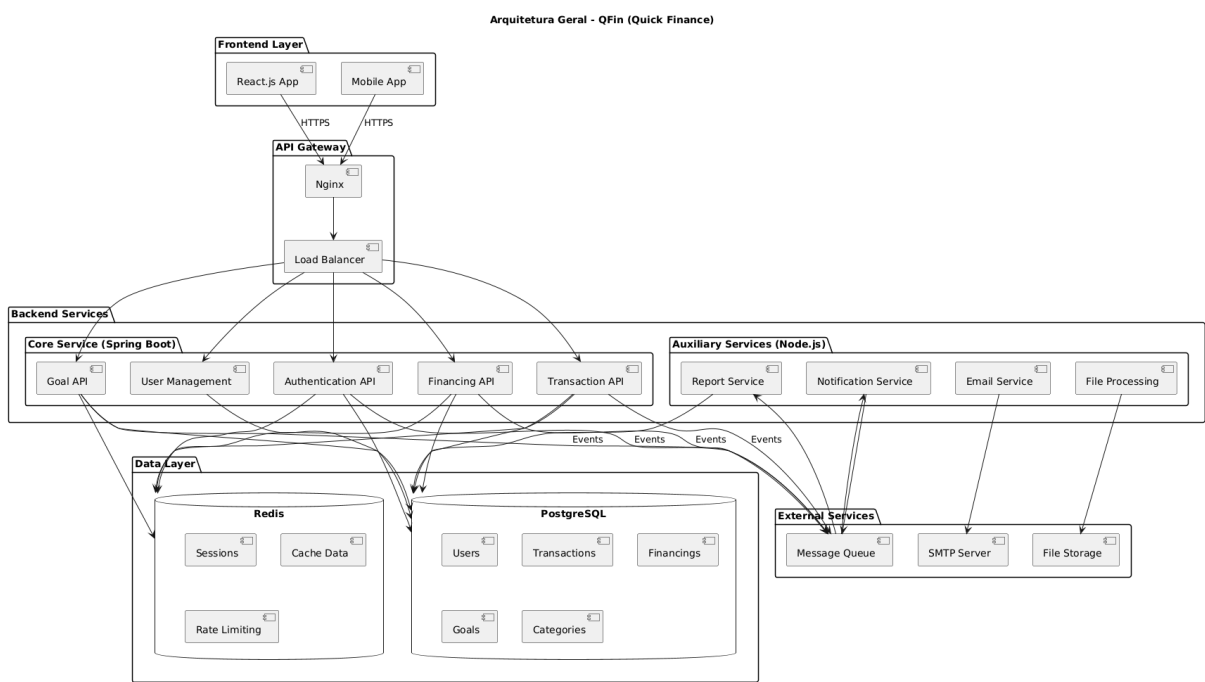


Figura 1: Diagrama da Arquitetura Geral do Sistema QFin.

7.2 Principais Componentes

A tabela a seguir resume os componentes chave da arquitetura:

Componente	Tecnologia	Responsabilidade
Frontend	React.js	Interface do usuário, interação e visualização de dados.
API Gateway	Nginx	Ponto único de entrada, roteamento, balanceamento

		de carga e terminação SSL.
Core Service	Spring Boot, Java 17	Lógica de negócio principal, APIs REST, autenticação e gestão de dados.
Auxiliary Services	Node.js	Tarefas assíncronas como notificações, relatórios e processamento de arquivos.
Banco de Dados	PostgreSQL	Armazenamento persistente dos dados relacionais.
Cache	Redis	Armazenamento de sessões, cache de dados e controle de rate limiting.
Message Queue	RabbitMQ / Kafka	Comunicação assíncrona entre os serviços de backend.
File Storage	AWS S3 / MinIO	Armazenamento de arquivos gerados, como relatórios e imagens.

7.3 Arquitetura em Camadas (Layered Architecture)

O Core Service (Spring Boot) é estruturado seguindo o padrão de arquitetura em camadas, que promove a separação de responsabilidades e o baixo acoplamento entre as diferentes partes do sistema.

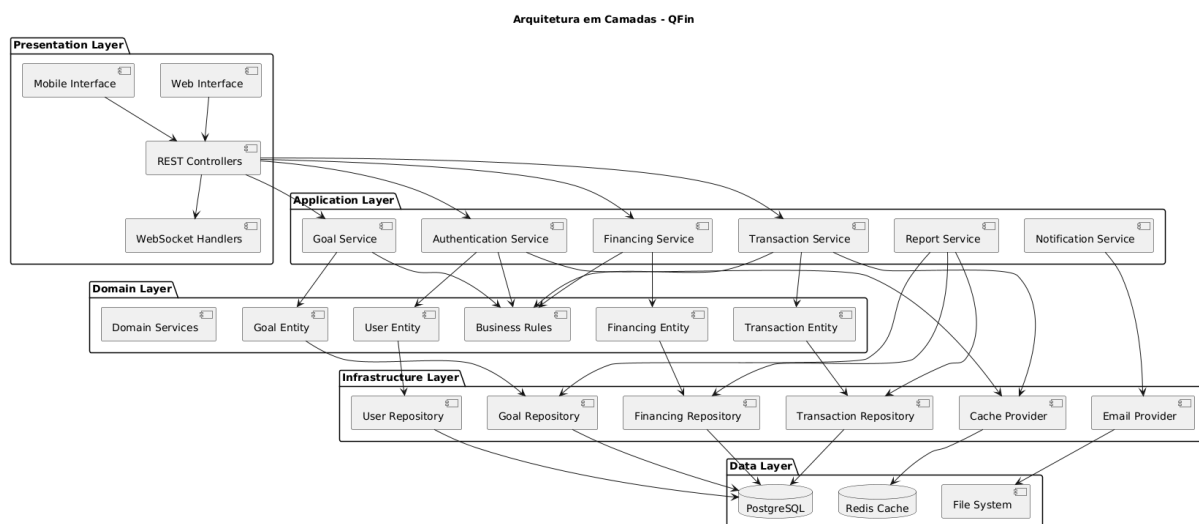


Figura 2: Diagrama da Arquitetura em Camadas do Core Service.

As camadas são definidas da seguinte forma:

1. Camada de Apresentação (Presentation Layer):

Responsável por expor as funcionalidades do sistema para o mundo externo. Inclui os REST Controllers que manipulam as requisições HTTP e os WebSocket Handlers para comunicação em tempo real.

2. Camada de Aplicação (Application Layer):

Orquestra as regras de negócio e as operações. É composta pelos Services (ex: TransactionService), que são chamados pelos controladores e contêm a lógica de aplicação, mas não as regras de negócio do domínio em si.

3. Camada de Domínio (Domain Layer):

O coração do software. Contém as Entities (ex: User , Transaction), que representam os conceitos do negócio, e as regras de negócio intrínsecas a essas entidades. É a camada mais isolada e independente.

4. Camada de Infraestrutura (Infrastructure Layer):

Implementa os detalhes técnicos para as outras camadas, como o acesso ao banco de dados (Repositories), comunicação com serviços externos (EmailProvider) e caching (CacheProvider).

7.4 Detalhamento dos Módulos

7.4.1. Core Service (Spring Boot)

Este é o serviço principal da aplicação, responsável por todas as operações síncronas e a lógica de negócio central. Ele é construído com Spring Boot e Java 17, utilizando Hibernate para a persistência de dados.

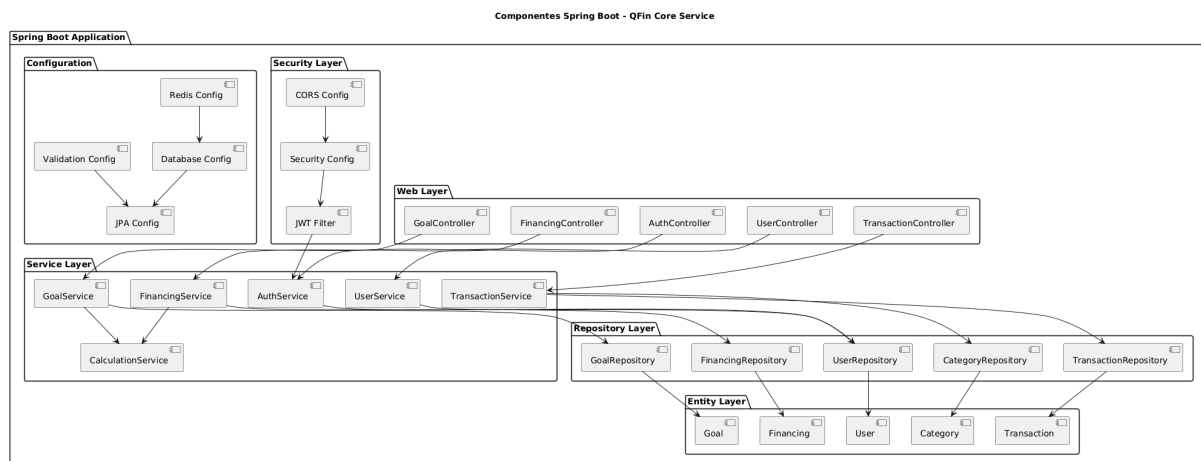


Figura 3: Diagrama de Componentes do Core Service (Spring Boot).

Os principais componentes internos são:

- Web Layer: Contém os `@RestController` que definem os endpoints da API REST. Utiliza o Spring MVC para mapear requisições HTTP para os métodos dos serviços.
- Security Layer: Implementada com Spring Security. Inclui filtros JWT para autenticação e autorização, configuração de CORS e proteção contra ataques comuns (CSRF, XSS).
- Service Layer: Onde a lógica de aplicação é implementada. Os `@Service` são transacionais e coordenam as interações com os repositórios e outras dependências.
- Repository Layer: Abstrai o acesso aos dados. Utiliza Spring Data JPA e Hibernate para definir interfaces (`@Repository`) que realizam as operações de CRUD no banco de dados PostgreSQL.
- Entity Layer: Contém as classes de domínio anotadas com JPA (`@Entity`) que são mapeadas para as tabelas do banco de dados.

7.4.2. Auxiliary Services (Node.js)

Para tarefas que podem ser executadas de forma assíncrona ou que se beneficiam do ecossistema Node.js, um conjunto de micro serviços é utilizado. Eles se comunicam com o serviço principal através de uma fila de mensagens (Message Queue).

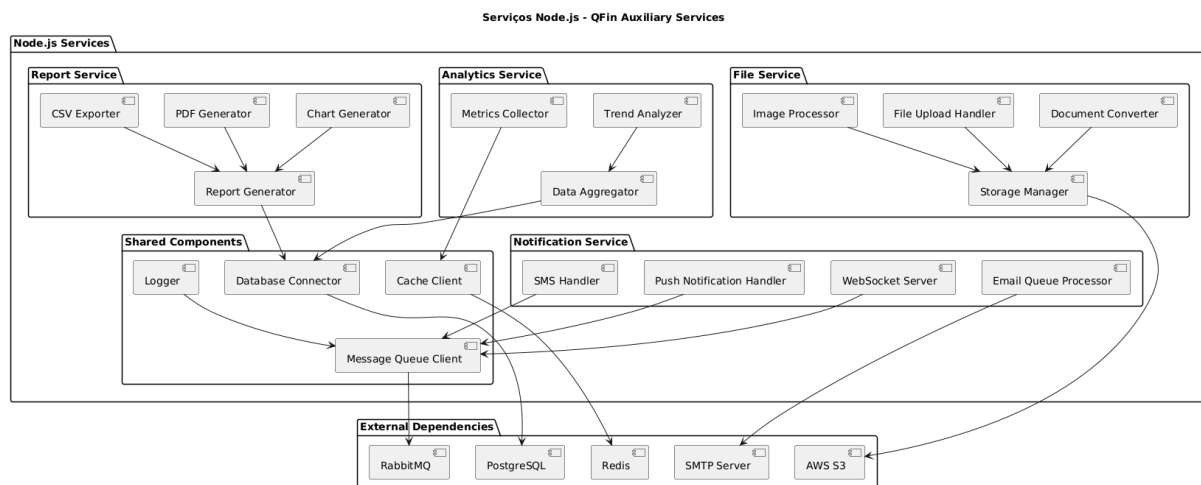


Figura 4: Diagrama dos Serviços Auxiliares (Node.js).

Os serviços incluem:

Notification Service: Ouve eventos da fila de mensagens e envia notificações para os usuários via WebSockets, Push Notifications ou Email.

Report Service: Gera relatórios complexos em segundo plano. Consome dados do PostgreSQL e utiliza bibliotecas como pdfkit e csv-writer para criar arquivos PDF e CSV.

File Service: Gerencia o upload, processamento e armazenamento de arquivos em um serviço de storage como o AWS S3.

7.5 Camada de Dados

A persistência de dados é um pilar fundamental da arquitetura, composta por PostgreSQL, Hibernate e Redis.

7.5.1. PostgreSQL e Hibernate PostgreSQL:

Escolhido por sua robustez, confiabilidade e suporte a funcionalidades avançadas de SQL. Ele armazena todos os dados relacionais do sistema.

Hibernate: Utilizado como a implementação do JPA (Java Persistence API) através do Spring Data JPA. Ele gerencia o mapeamento objeto-relacional (ORM), convertendo as entidades Java em registros no banco de dados e vice-versa. Isso simplifica as operações de banco de dados e abstrai a complexidade do SQL.

7.5.2. Redis

- Redis:

É um banco de dados em memória de alta performance, utilizado para:

- Cache de Sessão: Armazenar informações de sessão do usuário, permitindo que o Core Service seja stateless.
- Cache de Dados: Armazenar dados frequentemente acessados (ex: categorias, configurações) para reduzir a carga no PostgreSQL.
- Rate Limiting: Controlar o número de requisições por usuário para proteger a API contra abusos.

7.6 Padrões de Comunicação

7.6.1. Comunicação Síncrona:

REST API A comunicação entre o frontend e o backend, bem como entre alguns serviços, é feita através de APIs RESTful. O Spring Boot expõe endpoints que seguem as convenções REST, utilizando JSON como formato de dados.

7.6.2. Comunicação Assíncrona:

Message Queue Para desacoplar os serviços e melhorar a resiliência, um sistema de fila de mensagens é utilizado. O Core Service publica eventos na fila, e os serviços Node.js (consumidores) ouvem esses eventos para executar tarefas em segundo plano, como enviar um email de confirmação.

8. Diagramas de Classes Detalhados

Os diagramas estão divididos em quatro áreas principais para facilitar a compreensão:

1. Entidades JPA: O modelo de domínio persistente.
2. Serviços e Repositórios: A lógica de negócio e o acesso a dados.
3. Controladores REST: A camada de API que interage com o frontend.
4. DTOs (Data Transfer Objects): Os objetos que transportam dados entre as camadas.

8.1 Diagrama de Entidades JPA

Este diagrama representa o coração do sistema, o modelo de domínio. As classes aqui definidas são mapeadas diretamente para as tabelas do banco de dados PostgreSQL através do Hibernate/JPA. Elas contêm não apenas os dados, mas também as regras de negócio intrínsecas ao domínio.

Entidades JPA - QFin Domain Model



Figura 1: Diagrama das Entidades JPA Principais.

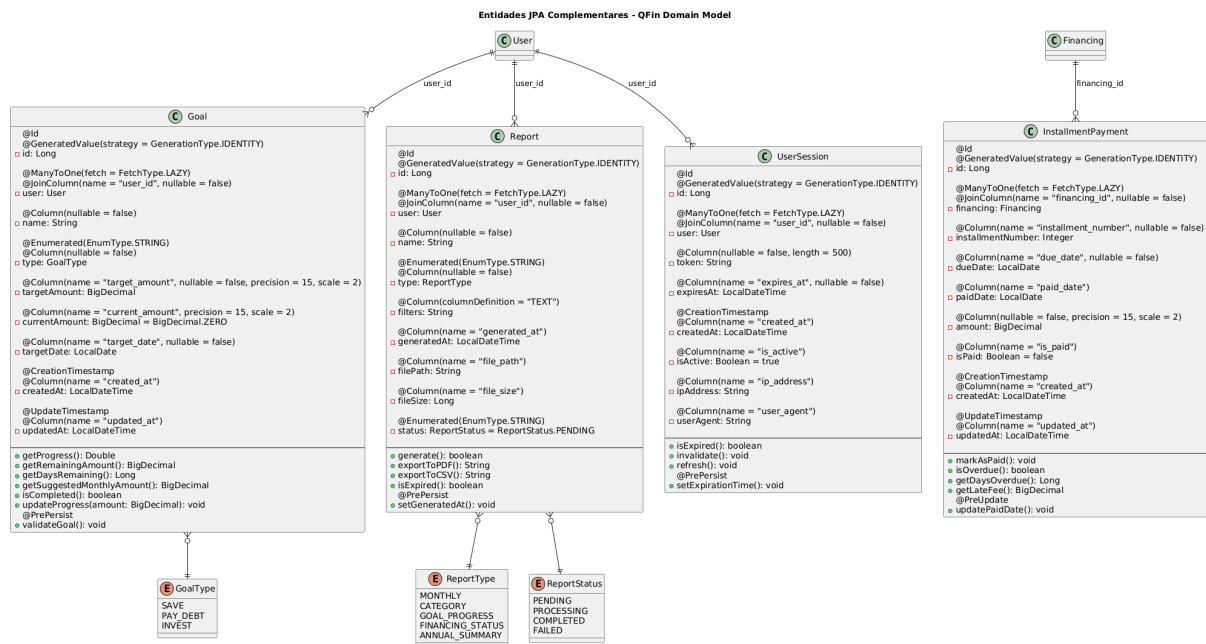


Figura 2: Diagrama das Entidades JPA Complementares

8.2 Diagrama de Serviços e Repositórios

Este diagrama ilustra a camada de aplicação e a camada de acesso a dados. Os Serviços contêm a lógica de negócio e orquestram as operações, enquanto os Repositórios abstraem a comunicação com o banco de dados.



Figura 3: Diagrama da Camada de Serviços e Repositórios.

8.3 Padrões de Design

- **Service Layer Pattern:** A lógica de negócio é encapsulada em classes de serviço (@Service), mantendo os controladores enxutos e focados em HTTP.
- **Repository Pattern:** O acesso aos dados é feito através de interfaces que estendem JpaRepository (@Repository). Isso abstrai a implementação da persistência e permite a fácil substituição ou teste.
- **Dependency Injection:** O Spring injeta as dependências (como repositórios em serviços) automaticamente usando a anotação @Autowired (ou, preferencialmente, injeção via construtor), promovendo baixo acoplamento.

8.4 Diagrama de Controladores REST

Este diagrama mostra a camada de apresentação, que expõe a API do sistema para o mundo exterior. Os controladores são responsáveis por receber requisições HTTP, chamar os serviços apropriados e retornar respostas formatadas.

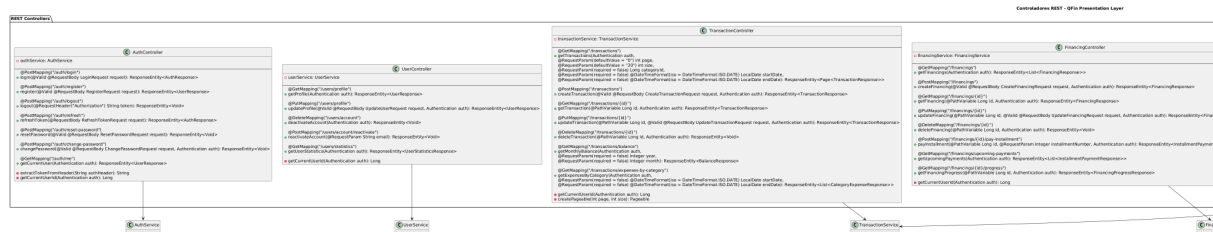


Figura 4: Diagrama da Camada de Controladores REST

8.4.1. Estrutura e Anotações

- **@RestController** : Marca a classe como um controlador REST, combinando **@Controller** e **@ResponseBody**.
- **@RequestMapping("/api/v1")** : Usado a nível de classe para versionar a API.
- **@GetMapping** , **@PostMapping** , **@PutMapping** , **@DeleteMapping** : Mapeiam os métodos para os verbos HTTP correspondentes.
- **@PathVariable** , **@RequestBody** , **@RequestParam** : Utilizados para extrair dados da URL, do corpo da requisição e dos parâmetros da query, respectivamente.
- **ResponseEntity** : Permite um controle total sobre a resposta HTTP, incluindo o corpo, os headers e o status code.
- **GlobalExceptionHandler** : Uma classe anotada com **@ControllerAdvice** que centraliza o tratamento de exceções, garantindo que a API sempre retorne respostas de erro padronizadas e consistentes.

8.5. Diagrama de DTOs (Data Transfer Objects)

Os DTOs são objetos simples que carregam dados entre as camadas, especialmente entre os controladores e os clientes da API. Eles são essenciais para desacoplar o modelo de domínio da representação externa, evitando a exposição de detalhes de implementação e permitindo a customização dos dados enviados e recebidos.

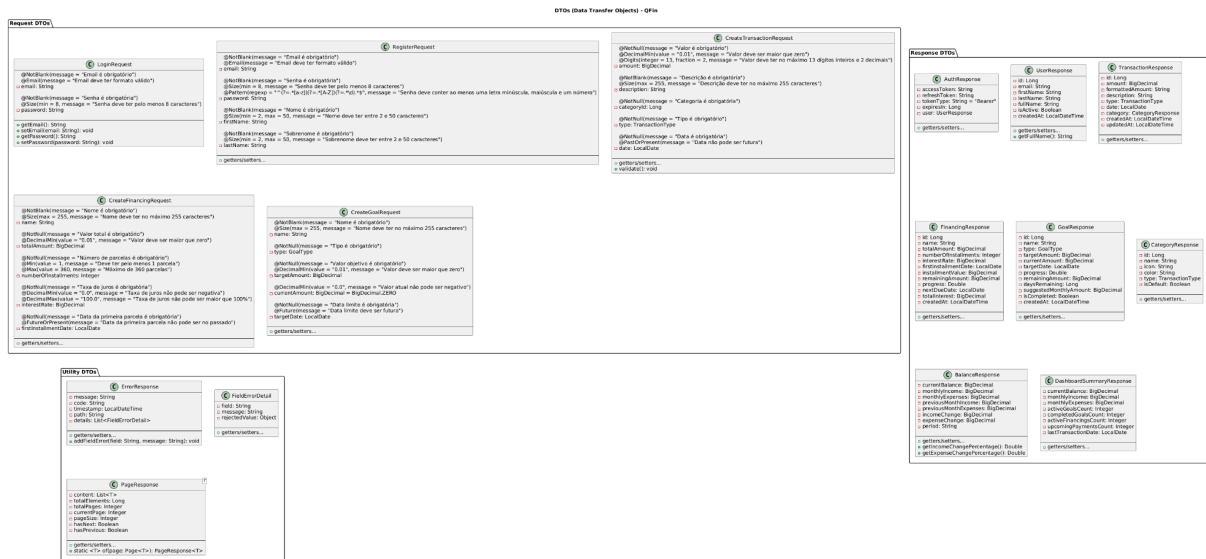


Figura 5: Diagrama dos DTOs.

8.5.1. Tipos de DTOs

- **Request DTOs:** Usados para receber dados nas requisições POST e PUT . Eles contêm anotações de validação (@NotBlank , @Size , @DecimalMin) do Bean Validation (JSR 380) para garantir a integridade dos dados na entrada da API.
- **Response DTOs:** Usados para formatar os dados enviados nas respostas da API. Eles são customizados para fornecer apenas as informações necessárias ao cliente, muitas vezes agregando dados de múltiplas entidades.
- **Utility DTOs:** Classes genéricas para respostas padronizadas, como ErrorResponse e PageResponse , que encapsula os resultados de uma paginação.

9. Diagramas de Sequência / Colaboração

Os diagramas foram criados com base nos casos de uso mais críticos e representativos do sistema, incluindo:

- Autenticação e Registro de Usuário
- Criação e Listagem de Transações
- Criação de Financiamentos e Metas
- Carregamento de Dados do Dashboard

9.1 Diagrama de Sequência - Login do Usuário

Este diagrama ilustra o fluxo de autenticação de um usuário no sistema. O processo envolve a validação de credenciais, a geração de tokens JWT e a criação de uma sessão de usuário.

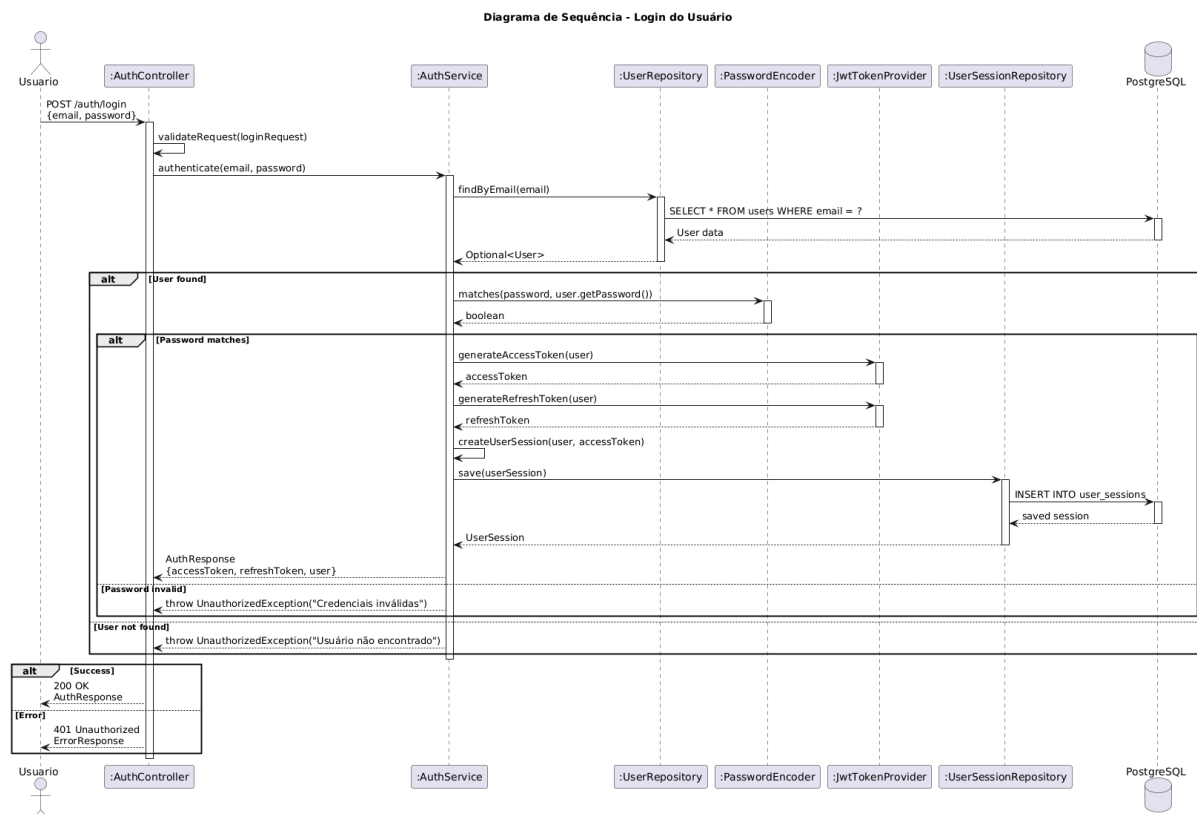


Figura 1: Fluxo de mensagens durante o processo de login.

9.1.1. Descrição do Fluxo

1. O Usuário envia uma requisição POST /auth/login com seu email e senha para o :AuthController.
2. O :AuthController invoca o :AuthService para realizar a autenticação.

3. O `:AuthService` busca o usuário no banco de dados através do `:UserRepository`.
4. Se o usuário existe, o `:AuthService` utiliza o `:PasswordEncoder` para comparar a senha fornecida com a senha armazenada (hash).
5. Se a senha é válida, o `:AuthService` chama o `:JwtTokenProvider` para gerar um `accessToken` e um `refreshToken`.
6. Uma nova sessão é registrada no `:UserSessionRepository`.
7. O `:AuthService` retorna os tokens e os dados do usuário para o `:AuthController`.
8. O `:AuthController` envia uma resposta 200 OK para o Usuário com os tokens de autenticação.

9.1.2. Pontos Chave Segurança:

- **Segurança:** A senha nunca é trafegada ou comparada em texto plano. O Spring Security utiliza um `PasswordEncoder` para garantir a segurança.
- **Stateless:** A utilização de JWT (JSON Web Tokens) permite que a API seja stateless. O estado da autenticação é mantido no cliente e validado a cada requisição.
- **Sessões Ativas:** O registro da sessão no banco de dados permite o monitoramento e a invalidação de sessões ativas, aumentando a segurança.

9.2 Diagrama de Sequência - Registro de Usuário

Este diagrama detalha o processo de criação de uma nova conta de usuário no sistema QFin.

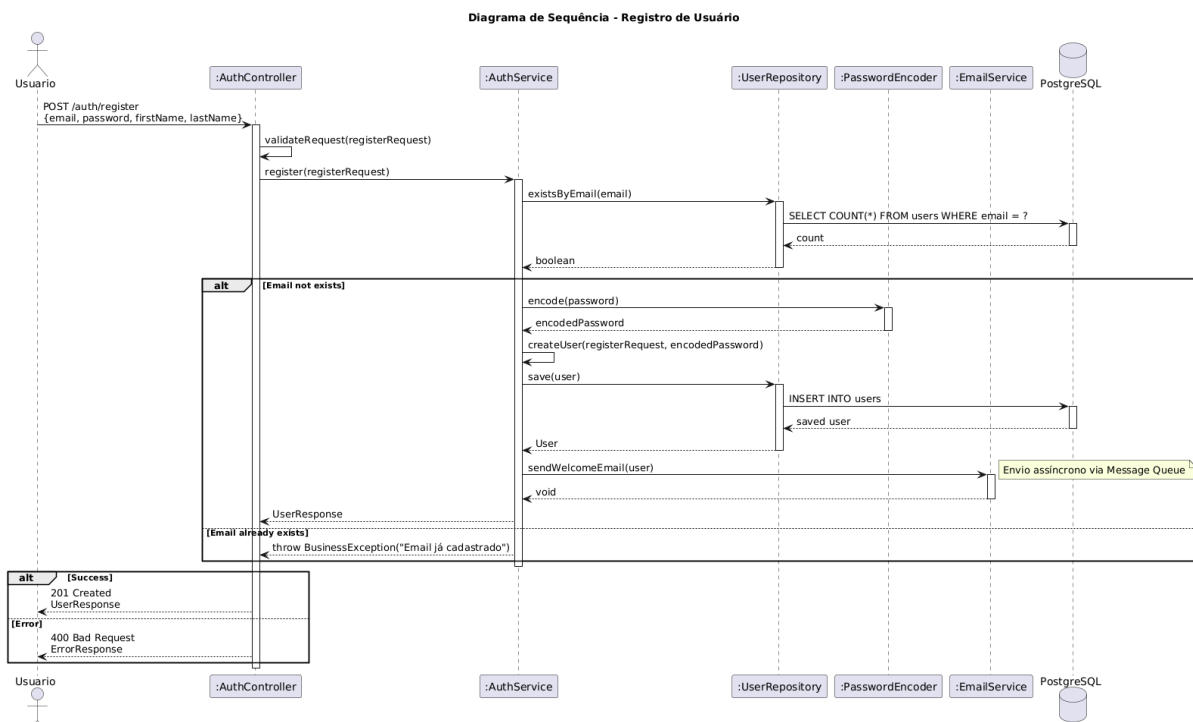


Figura 2: Fluxo de mensagens durante o registro de um novo usuário.

9.2.1. Descrição do Fluxo

1. O Usuário envia uma requisição POST /auth/register com seus dados (nome, email, senha).
2. O :AuthController chama o :AuthService para processar o registro.
3. O :AuthService primeiro verifica se o email já existe no sistema consultando o :UserRepository .
4. Se o email for único, a senha é codificada (hashed) usando o :PasswordEncoder .
5. Um novo objeto User é criado e salvo no banco de dados através do :UserRepository .
6. De forma assíncrona, o :AuthService dispara um evento para o :EmailService enviar um email de boas-vindas.
7. Uma resposta 201 Created com os dados do novo usuário é retornada.

9.3 Diagrama de Sequência - Criar Transação

Este diagrama mostra o fluxo de criação de uma nova transação financeira (receita ou despesa).

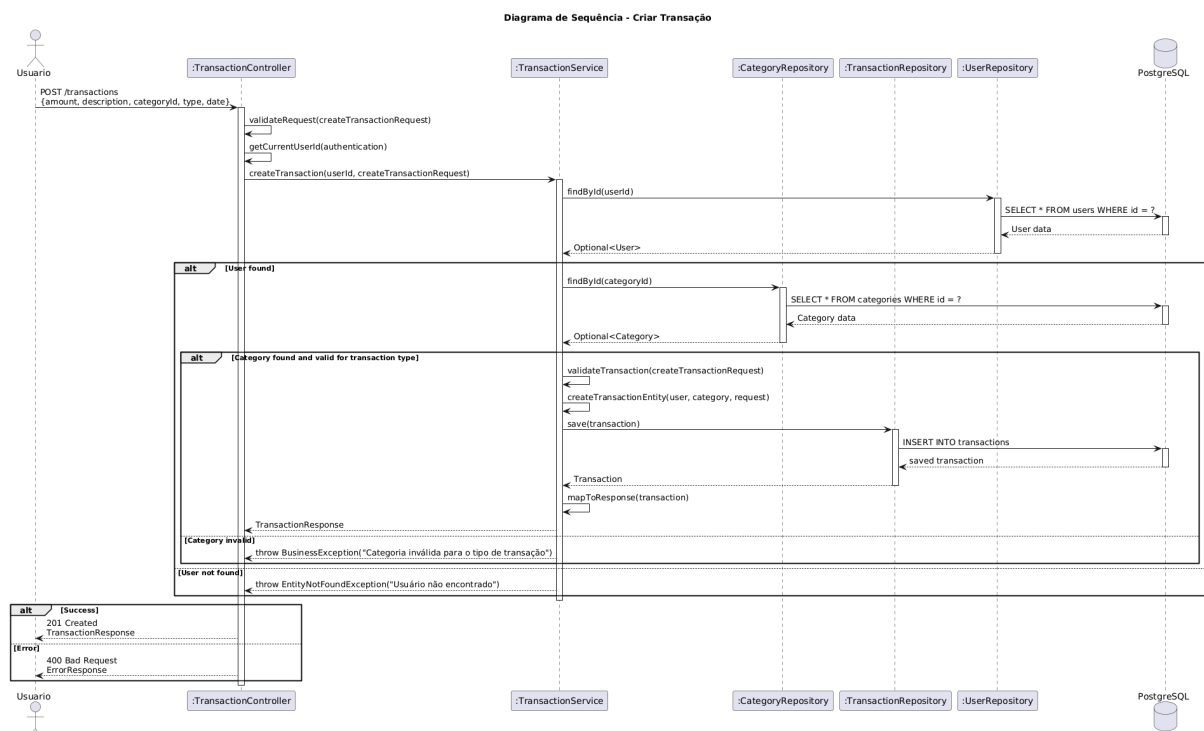


Figura 3: Fluxo de mensagens para a criação de uma nova transação.

9.3.1. Descrição do Fluxo

1. O Usuário (autenticado) envia uma requisição POST /transactions com os detalhes da transação.
2. O :TransactionController extrai o ID do usuário do token de autenticação e chama o :TransactionService .
3. O :TransactionService realiza as validações necessárias, como verificar se o usuário e a categoria informada existem (consultando :UserRepository e :CategoryRepository).
4. Se tudo for válido, uma nova entidade Transaction é criada e persistida no banco de dados através do :TransactionRepository .
5. A transação recém-criada é mapeada para um TransactionResponse (DTO) e retornada ao usuário com um status 201 Created.

9.4 Diagrama de Sequência - Listar Transações

Este diagrama ilustra como o sistema recupera e exibe a lista de transações de um usuário, incluindo suporte a filtros e paginação.

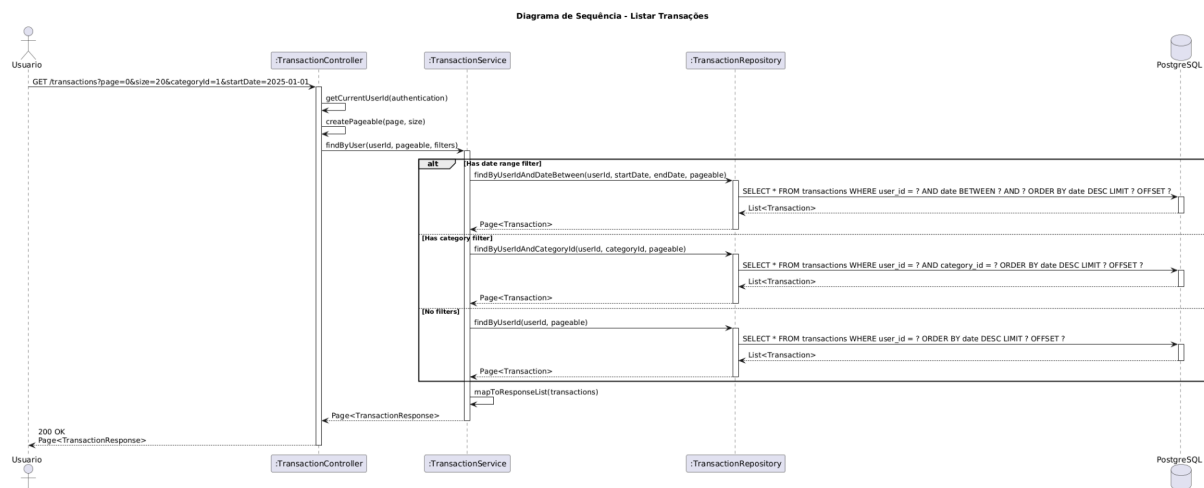


Figura 4: Fluxo de mensagens para a listagem de transações com filtros.

9.4.1. Descrição do Fluxo

1. O Usuário envia uma requisição GET /transactions com parâmetros opcionais de paginação e filtros (ex: categoryId , startDate).
2. O :TransactionController monta um objeto Pageable e repassa a chamada para o :TransactionService .
3. O :TransactionService seleciona o método apropriado no :TransactionRepository com base nos filtros fornecidos.
4. O :TransactionRepository executa uma query no banco de dados com as cláusulas WHERE , ORDER BY , LIMIT e OFFSET correspondentes.
5. O resultado é uma Page , que é mapeada para uma Page e retornada ao usuário.

9.5 Diagrama de Sequência - Criar Financiamento

Este diagrama detalha o processo complexo de cadastrar um novo financiamento, que envolve cálculos e a geração de um cronograma de parcelas.

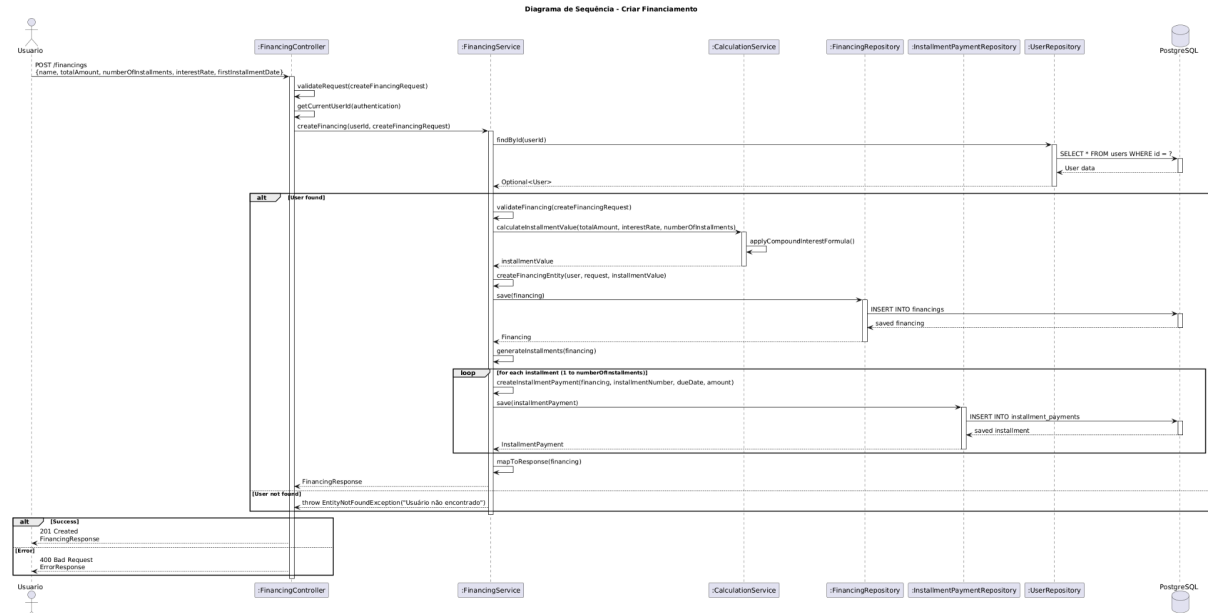


Figura 5: Fluxo de mensagens para a criação de um novo financiamento.

9.5.1 Descrição do Fluxo

1. O Usuário envia uma requisição POST /financings com os dados do financiamento.
2. O :FinancingController chama o :FinancingService .
3. O :FinancingService utiliza o :CalculationService para calcular o valor de cada parcela com base no valor total, taxa de juros e número de parcelas.
4. A entidade Financing é criada e salva no banco de dados.
5. Em seguida, o serviço entra em um loop para criar e salvar todas as entidades InstallmentPayment associadas ao financiamento, uma para cada parcela.
6. A resposta final contém os detalhes do financiamento e um resumo do cronograma.

9.7 Diagrama de Sequência - Carregar Dashboard

Este diagrama mostra como os dados consolidados do dashboard são carregados, envolvendo múltiplas consultas em paralelo para otimizar a performance.

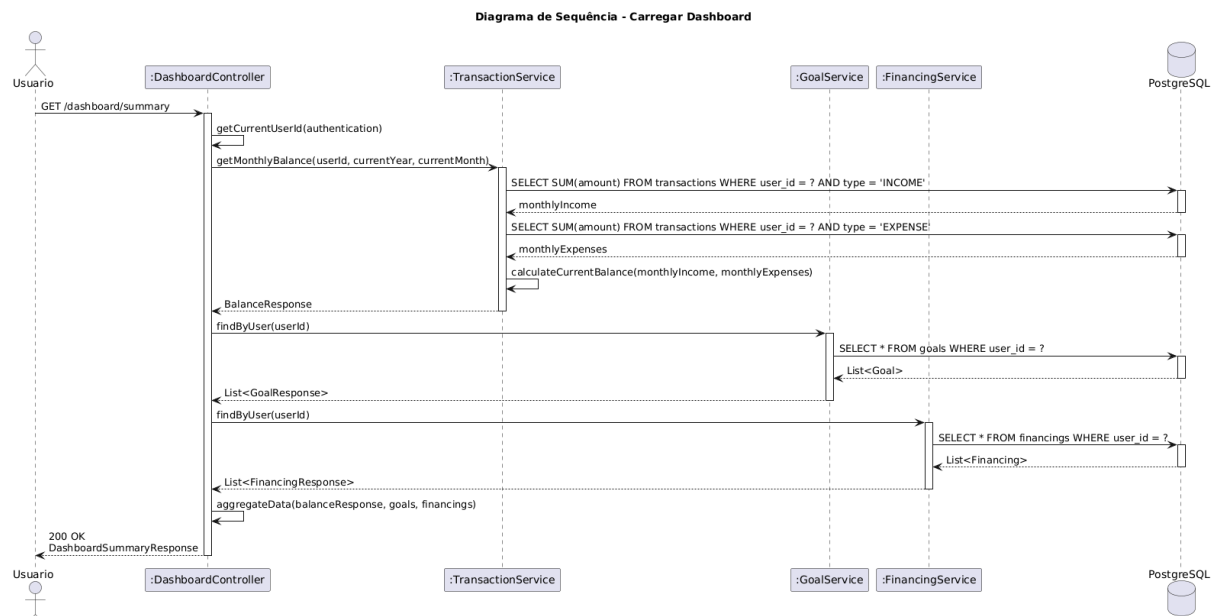


Figura 6: Fluxo de mensagens para o carregamento dos dados do dashboard.

9.7.1 Descrição do Fluxo

1. O Usuário acessa a página do dashboard, disparando uma requisição GET /dashboard/summary .
2. O :DashboardController orquestra a busca de dados de diferentes serviços.
3. Utilizando o fragmento par (paralelo), o controlador faz chamadas simultâneas para: :TransactionService para obter o balanço mensal. :GoalService para obter o progresso das metas ativas. :FinancingService para obter os financiamentos em andamento.
4. Cada serviço consulta seu respectivo repositório, que por sua vez busca os dados no banco de dados.
5. Após receber todas as respostas, o :DashboardController agrega os dados em um único DTO (DashboardSummaryResponse) e o retorna para o usuário.

10. Modelo de Dados (DER – Diagrama Entidade-Relacionamento)

Os diagramas a seguir ilustram a estrutura do banco de dados. O primeiro oferece uma visão geral das entidades e seus relacionamentos, enquanto o segundo fornece um detalhamento completo, incluindo tipos de dados, constraints e índices.

10.1 Visão Geral do Modelo de Dados

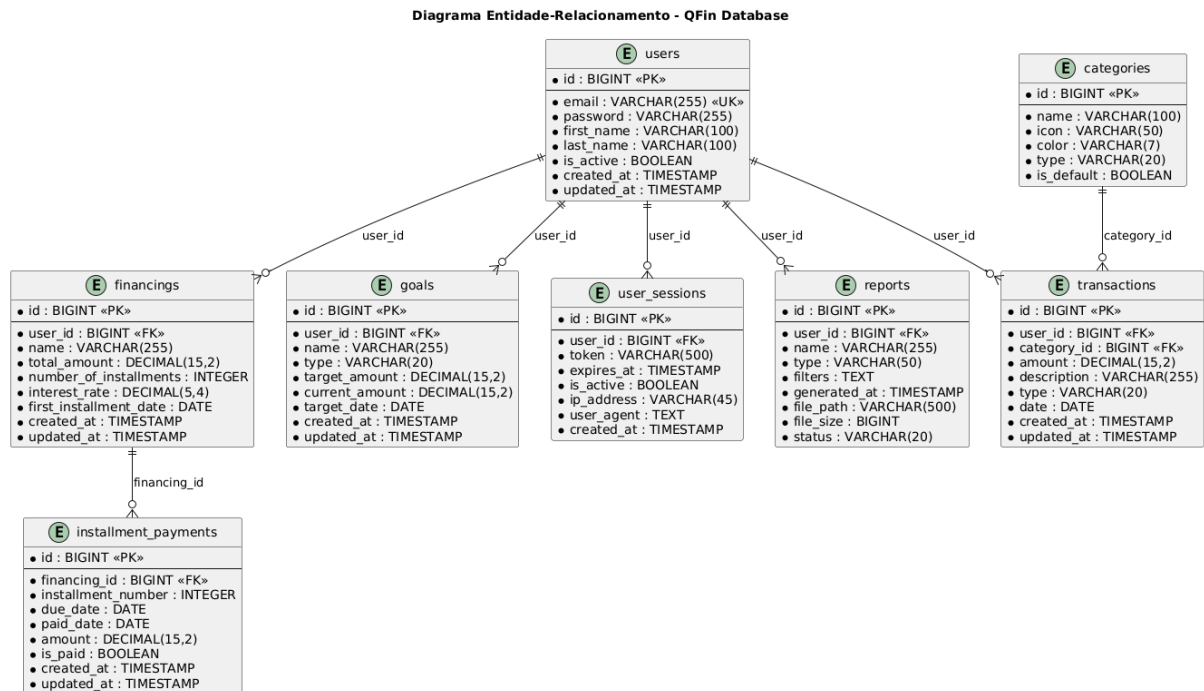


Figura : Diagrama Entidade-Relacionamento principal do sistema QFin.

10.2 Modelo de Dados Detalhado com Constraints e Índices

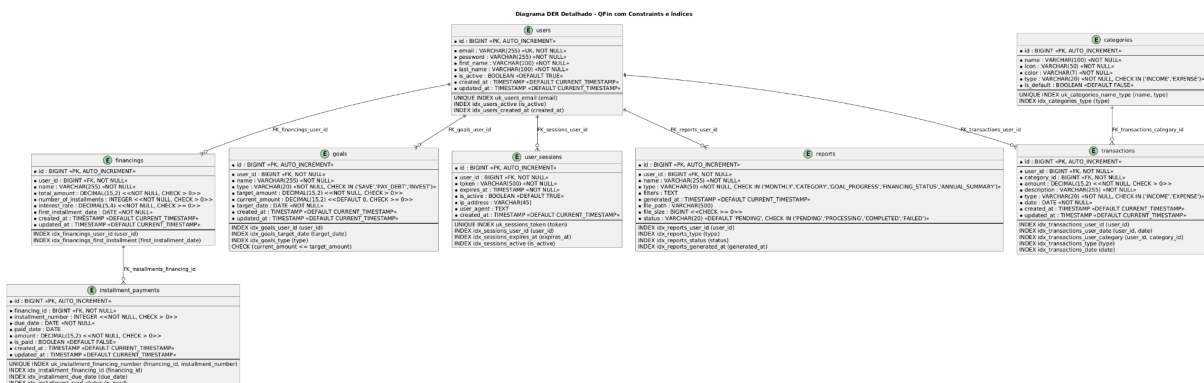


Figura : Diagrama Entidade-Relacionamento detalhado com constraints e índices.

10.3 Dicionário de Dados

Esta seção descreve em detalhes cada uma das tabelas que compõem o banco de dados do QFin.

10.3.1 Tabela users

Armazena as informações de todos os usuários cadastrados no sistema. É a entidade central do modelo, à qual a maioria das outras tabelas se relaciona.

Coluna	Tipo de Dado	Constraints	Descrição
id	BIGSERIAL	PK	Identificador único do usuário.
email	VARCHAR(255)	UK, NOT NULL	Email único do usuário, utilizado para login.
password	VARCHAR(255)	NOT NULL	Senha do usuário, armazenada em formato de hash (BCrypt).
first_name	VARCHAR(100)	NOT NULL	Primeiro nome do usuário.
last_name	VARCHAR(100)	NOT NULL	Sobrenome do usuário.
is_active	BOOLEAN	DEFAULT TRUE	Indica se a conta do usuário está ativa.
created_at	TIMESTAMP	DEFAULT NOW()	Data e hora de criação do registro.
updated_at	TIMESTAMP	DEFAULT NOW()	Data e hora da última atualização do registro.

10.3.2 Tabela categories

Define as categorias que podem ser utilizadas para classificar as transações, permitindo uma melhor organização financeira.

Coluna	Tipo de Dado	Constraints	Descrição
id	BIGSERIAL	PK	Identificador único da categoria.
name	VARCHAR(100)	NOT NULL	Nome da categoria (ex: "Alimentação", "Salário")

icon	VARCHAR(50)	NOT NULL	Nome do ícone associado à categoria.
color	VARCHAR(7)	NOT NULL	Cor em formato hexadecimal (ex: "#FF") para categoria.
type	VARCHAR(20)	NOT NULL, CHEK	Tipo da categoria: 'INCOME' (Receita) ou 'EXPE' (Despesa).
is_default	BOOLEAN	DEFAULT FALSE	Indica se é uma categoria padrão do sistema.

10.3.3 Tabela transactions

Registra todas as movimentações financeiras (receitas e despesas) de cada usuário.

Coluna	Tipo de Dado	Constraints	Descrição
id	BIGSERIAL	PK	Identificador único da transação.
user_id	BIGINT	FK (users), NOT NULL	Chave estrangeira que referencia o usuário que fez a transação.
category_id	BIGINT	FK(categories), NOT NULL	Chave estrangeira que referencia a categoria.
amount	DECIMAL(15,2)	NOT NULL, CHECK>0	Valor da transação.
description	VARCHAR(255)	NOT NULL	Descrição da transação.
type	VARCHAR(20)	NOT NULL, CHECK	Tipo da transação: 'INCOME' ou 'EXP'
date	DATE	NOT NULL	Data em que a transação ocorreu.

10.3.4 Tabela financings

Armazena os dados de financiamentos e empréstimos contratados pelos usuários.

Coluna	Tipo de Dado	Constraints	Descrição
id	BIGSERIAL	PK	Identificador único do financiamento.
user_id	BIGINT	FK (users), NOT NULL	Chave estrangeira que referencia o usuário.
name	VARCHAR(255)	NOT NULL	Nome ou descrição do financiamento.
total_amount	DECIMAL(15,2)	NOT NULL, CHECK > 0	Valor total do financiamento.
number_of_installments	INTEGER	NOT NULL, CHECK > 0	Número total de parcelas.
interest_rate	DECIMAL(5,4)	NOT NULL, CHECK >= 0	Taxa de juros mensal em formato decimal (ex: 0.0150 para 1.5%).
first_installment_date	DATE	NOT NULL	Data de vencimento da primeira parcela.

10.3.5 Tabela installment_payments

Controla o status de cada parcela individual de um financiamento.

Coluna	Tipo de Dado	Constraints	Descrição
id	BIGSERIAL	PK	Identificador único da parcela.
financing_id	BIGINT	FK (financings), NOT NULL	Chave estrangeira que referencia o financiamento.
installment_number	INTEGER	NOT NULL, CHECK > 0	Número da parcela (1, 2, 3...).
due_date	DATE	NOT NULL	Data de vencimento da parcela.
paid_date	DATE		Data em que a

			parcela foi paga. NULL se não foi paga.
amount	DECIMAL(15,2)	NOT NULL, CHECK > 0	Valor da parcela.
is_paid	BOOLEAN	DEFAULT FALSE	Indica se a parcela já foi paga.

10.3.6 Tabela goals

Gerencia as metas financeiras criadas pelos usuários.

Coluna	Tipo de Dado	Constraints	Descrição
id	BIGSERIAL	PK	Identificador único da meta.
user_id	BIGINT	FK (users), NOT NULL	Chave estrangeira que referencia o usuário.
name	VARCHAR(255)	NOT NULL	Nome da meta (ex: "Viagem para a Europa").
type	VARCHAR(20)	NOT NULL, CHECK	Tipo da meta: 'SAVE', 'PAY_DEBT', 'INVEST'.
target_amount	DECIMAL(15,2)	NOT NULL, CHECK > 0	Valor objetivo da meta.
current_amount	DECIMAL(15,2)	DEFAULT 0, CHECK >= 0	Valor atualmente acumulado para a meta.
target_date	DATE	NOT NULL	Data limite para alcançar a meta.

11. Testes

Tela / Módulo	Campo ou Função Testada	Descrição do Teste Realizado	Resultado	Falha Encontrada	Proposta de Correção / Observação
Criar Conta	Nome Completo	Teste de preenchimento obrigatório e aceitação de caracteres	✓	Nenhuma	Campo validado corretamente.
Criar Conta	E-mail	Teste com formato válido e inválido ('teste@teste.com', 'teste.com')	✓	Nenhuma	Validação de formato funcional.
Criar Conta	Senha	Teste de mínimo de 6 caracteres e correspondência com campo "Confirmar Senha"	✓	Nenhuma	Regras aplicadas corretamente.
Criar Conta	Botão "Criar Conta"	Teste de envio com campos válidos e inválidos	✓	Nenhuma	Mensagens de erro exibidas corretamente.
Criar Conta	Link "Entrar"	Redirecionamento para tela de login	✓	Nenhuma	Navegação correta.
Dashboard	Exibição de Saldo Total, Receitas, Despesas e Financiamentos	Verificação de atualização dinâmica dos valores	✓	Nenhuma	Dados atualizam corretamente após transações.
Dashboard	Gráficos (Despesas por Categoria e Receitas vs Despesas)	Renderização e atualização de gráficos com dados reais	✓	Nenhuma	Exibição correta e responsiva.
Dashboard	Adicionar Financiamento	Inserção de novo item com valor, tipo e prazo	✓	Nenhuma	Barras de progresso e parcelas exibidas corretamente.
Dashboard	Cálculo de Progresso	Teste de atualização de percentual e valores restantes	✓	Nenhuma	Cálculo exibido corretamente.
Transações	Adicionar Receita / Despesa	Inserção de nova transação com valores positivos e negativos	✓	Nenhuma	Saldo atualizado corretamente.
Transações	Filtros (Todas / Receitas / Despesas)	Troca de aba e exibição dos registros filtrados	✓	Nenhuma	Filtros funcionando corretamente.
Transações	Excluir Transação	Remoção de item e atualização do saldo total	✓	Nenhuma	Atualização imediata do valor total.
Perfil	Exibição de Dados Pessoais	Nome, e-mail e senha protegida	✓	Nenhuma	Exibição correta dos dados cadastrados.
Perfil	Alterar Senha	Teste de alteração e salvamento de nova senha	✓	Nenhuma	Atualiza corretamente e mantém segurança.
Perfil	Notificações / Preferências	Acesso à configuração de notificações	✓	Nenhuma	Opção acessível e funcional.
Perfil	Sair da Conta	Logout do usuário e redirecionamento para tela inicial	✓	Nenhuma	Sessão encerrada corretamente.
Financiamentos	Adicionar Financiamento	Ao clicar, deve abrir o formulário/modal de novo financiamento	X	falha	Revisar o evento